

따라하기

aidot-express 로 웹서버 만들기

관리자 콘솔에서 클릭만으로 "우리 반 간식 목록" API 만들기

코딩을 한 줄도 몰라도 됩니다. 버튼만 누르면 진짜 API 가 생깁니다.

aidot-express · Author: Mike



파트 1 : 따라하면서 만들어보기

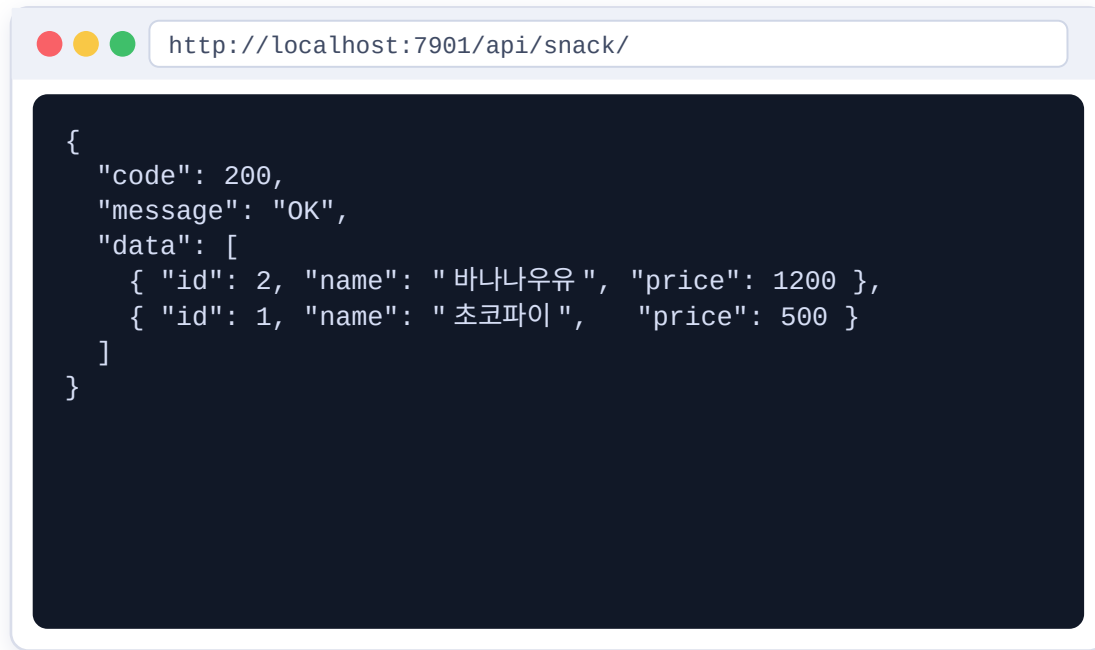
오늘 만들 것

" 간식 이름과 값을 저장하고 꺼내 보는 " 작은 웹서버 기능 5 개

만들 주소 (API) 5 개

GET	/api/snack/	간식 전체 보기
GET	/api/snack/1	1 번 간식만 보기
POST	/api/snack/	새 간식 추가하기
PUT	/api/snack/1	1 번 간식 고치기
DELETE	/api/snack/1	1 번 간식 지우기

브라우저에서 첫 번째 주소를 열면 이렇게 보여요



이 글자 덩어리를 JSON 이라고 불러요 .
프로그래머 이야기할 때 사용하는 " 글의 모양 " 이에요 .

순서 ① 테이블 만들기 → ② SQL → ③ 서비스 → ④ 컨트롤러 → ⑤ 테스트

설치 ① — 프로젝트 폴더를 받았을 때

소스 폴더 (aidot-express) 를 통째로 받은 경우

1 Node.js 가 깔려 있는지 확인하기

명령 프롬프트

```
C:\work\aidot-express> node -v  
v22.14.0 ← v20.19 이상이면 OK
```

안 나오면 nodejs.org 에서 LTS 를 받아 설치하세요 .

2 MariaDB 를 설치하고 .env 파일 만들기

아래 명령어로 만들어요 , 그리고 DB 접속 정보는 직접 수정해야 해요 . (.env 파일 열어 수정)

명령 프롬프트

```
C:\work\aidot-express> cp .env.example .env
```

3 npm start 명령어로 실행하기

명령어를 입력하면 실행돼요 .

명령 프롬프트

```
C:\work\aidot-express> npm start
```

2-2 메모장에서 .env 파일 열어 수정하기

.env 파일

```
DB_TYPE=mariadb  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_USER=root  
DB_PASSWORD=admin  
DB_DATABASE=aidot_express
```

명령 프롬프트

```
C:\work\aidot-express> npm start  
[start] Node.js v22.14.0 OK  
[start] 루트 의존성 최신 상태  
[start] 관리자 콘솔 (admin-client/dist) 최신 상태  
[DB] mariadb 접속 확인 OK  
[migration] 완료 - 적용 0 건  
서버 기동 완료 http://localhost:7901
```

4

http://localhost:7901/

관리자 콘솔 로그인 화면
(로그인 admin / admin1234)

설치 ② — 설치 파일 (.exe) 을 받았을 때

검은 창도, 명령어도 필요 없어요. 두 번 클릭하고 트레이 아이콘만 찾으면 됩니다

1

Setup 파일을 두 번 클릭

Aidot Express-Setup-1.32.0-x64.exe → " 추가 정보 → 실행 " → 다음 → 설치

2

설치 마법사에서 포트·DB 를 입력

기본값 7901 / 7902 그대로 두어도 됩니다. DB 는 mariadb

3

설치가 끝나면 저절로 실행됨

시작 화면이 잠깐 뜨고, 서버가 뒤에서 켜집니다. 창은 안 보여도 정상이에요!

4

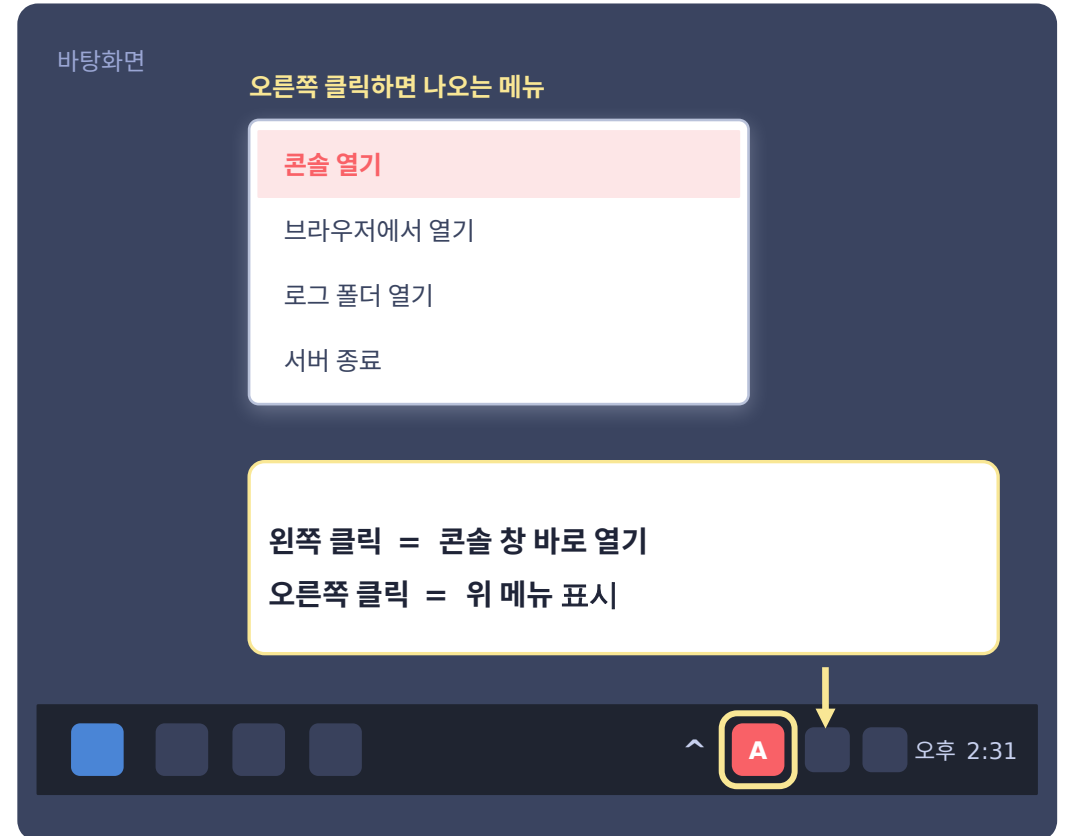
오른쪽 아래 트레이 아이콘 찾기

안 보이면 시계 왼쪽의 ^ (숨겨진 아이콘 표시) 를 눌러 보세요.

창의 X 를 눌러도 꺼지지 않고 트레이로 숨어요.

정말 끄려면 트레이 → [서버 종료] 를 눌러야 합니다.

작업표시줄 오른쪽 아래



설치

정리 : Setup 실행 → 포트·DB 입력 → 설치 → 트레이 아이콘 클릭 → 콘솔 열기 (로그인 admin / admin1234)

시작하기 전에 — 준비물 3 가지

앞의 따라하기 1 번 또는 2 번을 끝냈다면 아래 1~2 번은 이미 되어 있으니 3 번만 하면 돼요

1

서버를 켜 두기

검은 창 (명령 프롬프트) 에서 `npm start` 를 치고 켜 둔 채로 두세요 .

```
C:\work\aidot-express> npm start
```

2

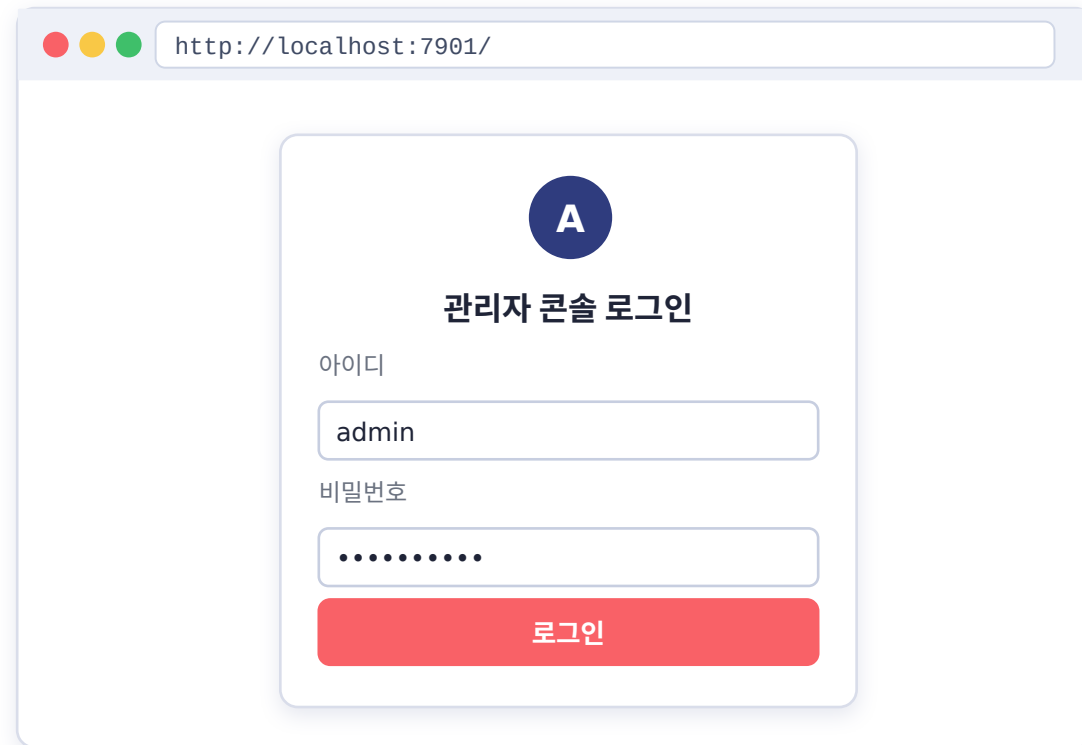
콘솔에 로그인하기

브라우저 주소창에 `localhost:7901` → 아이디 `admin` / 비밀번호 `admin1234`

3

간식을 담은 “테이블” 을 하나 만들기

테이블이 있어야 저장할 곳이 생겨요 . 만드는 방법은 다음 페이지를 보세요



처음 들어가면 위쪽에 비밀번호를 바꾸라는 노란 띠가 떠요 →
[설정 → 내 정보 → 비밀번호 변경] 에서 꼭 바꾸세요 !

확인

준비 확인 : 검은 창에 서버가 켜져 있고 · 콘솔에 로그인되고 · 테이블을 만들 준비가 됐다면 이제 출발할 수 있어요 .

콘솔 화면 구경하기

로그인 - 기본 관리자 계정으로 로그인하세요

**aidot-express**
Spring-style Node.js Framework

구조는 Spring처럼, 속도는 번개처럼

- ✔ Spring 식 계층 구조 (Controller / Service / SQL)
- ✔ 서버 재시작 없는 핫 리로드
- ✔ 콘솔 내장 — SQL · 서비스 · API 테스트
- ✔ DB + 미들웨어(전문) 컨트롤러를 한 곳에서



```
graph TD; HTTP[HTTP 요청] --> Controller["@Controller<br/>라우트 매핑"]; Controller --> Service["@Service<br/>비즈니스 로직"]; Service --> Sql["@Sql<br/>쿼리 주입"]; Sql --> DB[(DB)];
```

Node.js · Express · Annotation based Routing



English **한국어**

로그인

개발도구 콘솔에 접속하려면 로그인하세요

사용자 ID

비밀번호

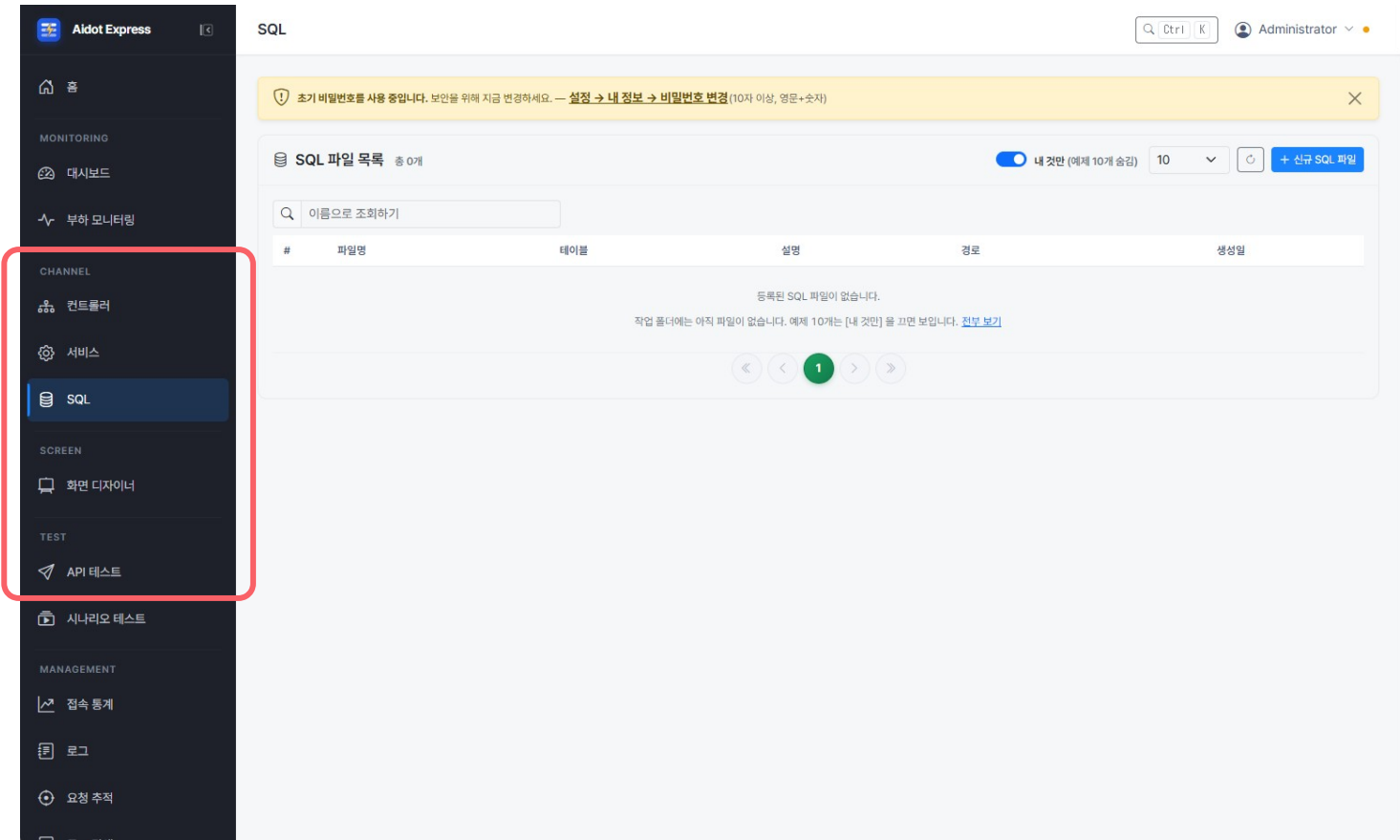
☐ 아이디 저장

로그인

DB 정상

콘솔 화면 구경하기

왼쪽 메뉴들은 메뉴 그룹 (모니터링 · 채널 · 화면 · 테스트 · 관리 등) 으로 나뉘어 있어요 . 오늘은 이 중에서 4 개만 사용합니다



오늘 사용할 메뉴

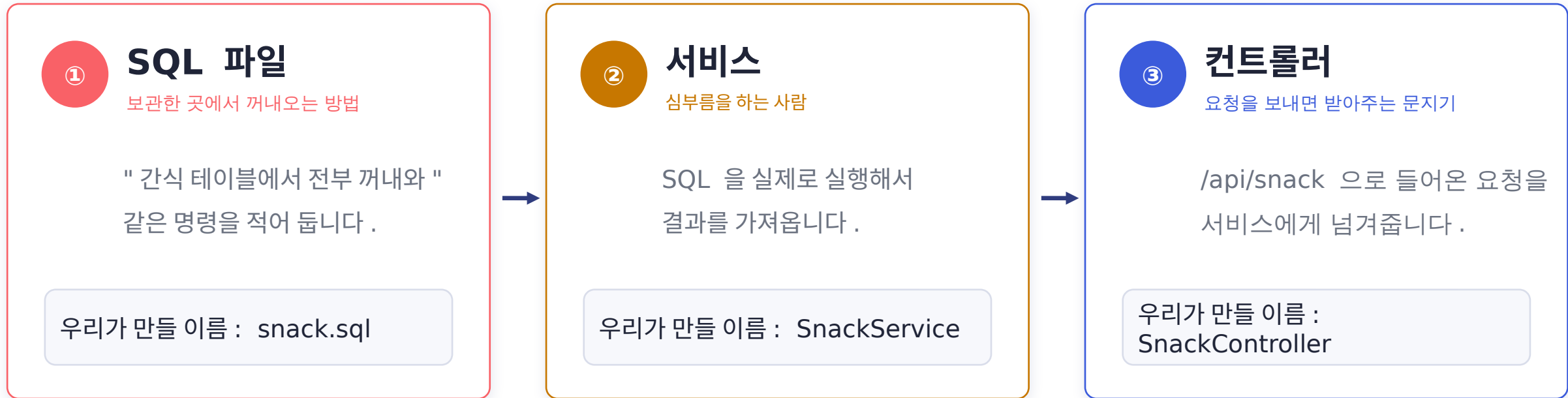
- 1 SQL**
저장소에서 자료를 꺼내는 방법을 적는 곳
- 2 서비스**
SQL 을 불러다 쓰는 심부름꾼
- 3 컨트롤러**
주소 (URL) 로 요청을 보내면 받아주는 문지기
- 4 API 테스트**
만든 주소가 잘 되는지 눌러보는 곳

요령

메뉴는 위에서부터 훑지 말고 만드는 순서대로 **SQL → 서비스 → 컨트롤러** 로 옮겨 다니세요 .

만들기 전에 — 3 층 구조만 이해하면 끝

학교 매점에 비유해볼까요



요령

만드는 순서도 똑같이 ① SQL → ② 서비스 → ③ 컨트롤러 입니다 . 앞의 것이 있어야 뒤의 것을 고를 수 있어요 .

0 단계 — 간식을 담은 " 테이블 " 만들기

HeidiSQL(MariaDB 를 깔면 같이 설치됨) 에서 딱 한 번만 하면 됩니다

1

HeidiSQL 을 열고 DB 에 접속한 후 위쪽 [쿼리] 탭을 누릅니다

2

왼쪽 트리에서 aidot-app 을 선택한 후
아래 글자를 그대로 복사해서 붙여넣습니다

```
CREATE TABLE snack (  
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  name        VARCHAR(50)  NOT NULL,  
  price       INT           NOT NULL DEFAULT 0,  
  memo        VARCHAR(200) NULL,  
  created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (id)  
)
```

3

실행 아이콘을 누릅니다 → 아래에 1 줄 성공한 것으로 나오면
됩니다
왼쪽 트리의 **aidot-app** 안에 **snack** 테이블이 보입니다

이런 테이블을 만들 거예요

snack (간식 테이블)

id	name	price	memo
1	초코파이	500	제일 인기
2	바나나우유	1200	체육시간 뒤에
...			

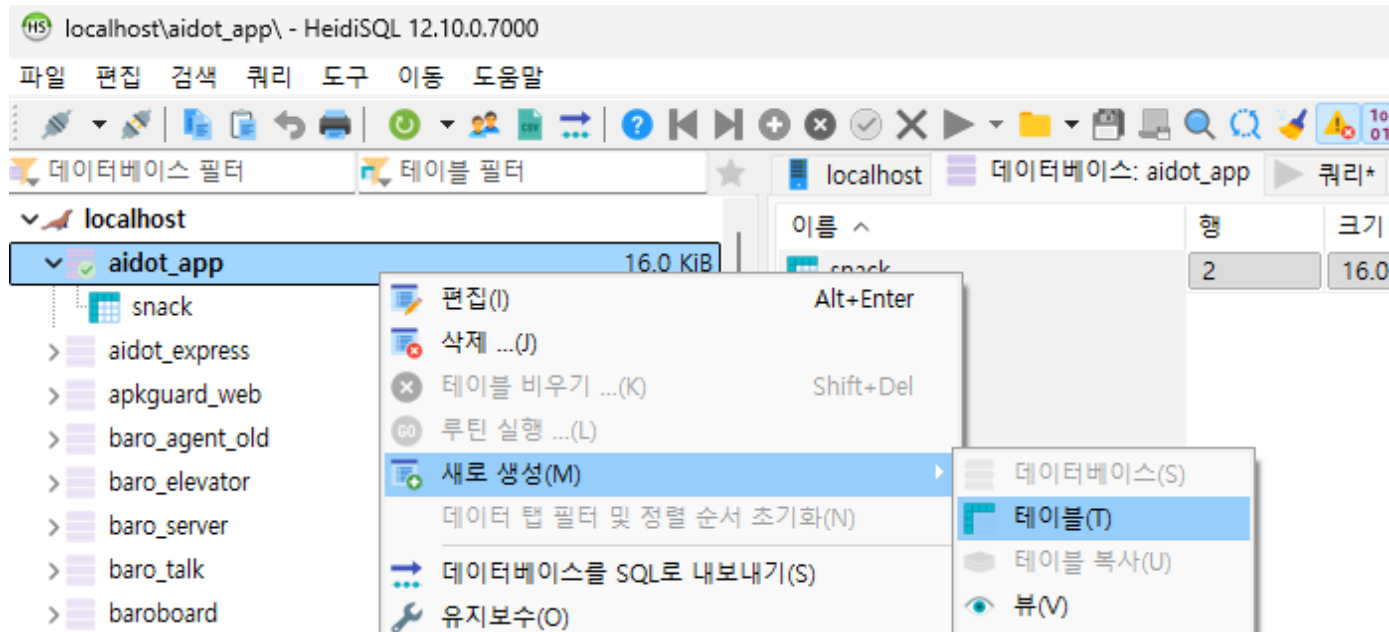
id 는 새로 넣을 때마다 1, 2, 3... 자동으로 붙어요

SQLite 를 쓰고 있다면 ?

같은 SQL 문을 SQLite 도구 (DB Browser for SQLite) 에
붙여넣고 실행합니다

0 단계 다른 방법 ① — 화면으로 테이블 만들기

SQL 을 적는 게 어렵다면 , HeidiSQL 화면에서 마우스로 만들 수 있어요



① 왼쪽에서

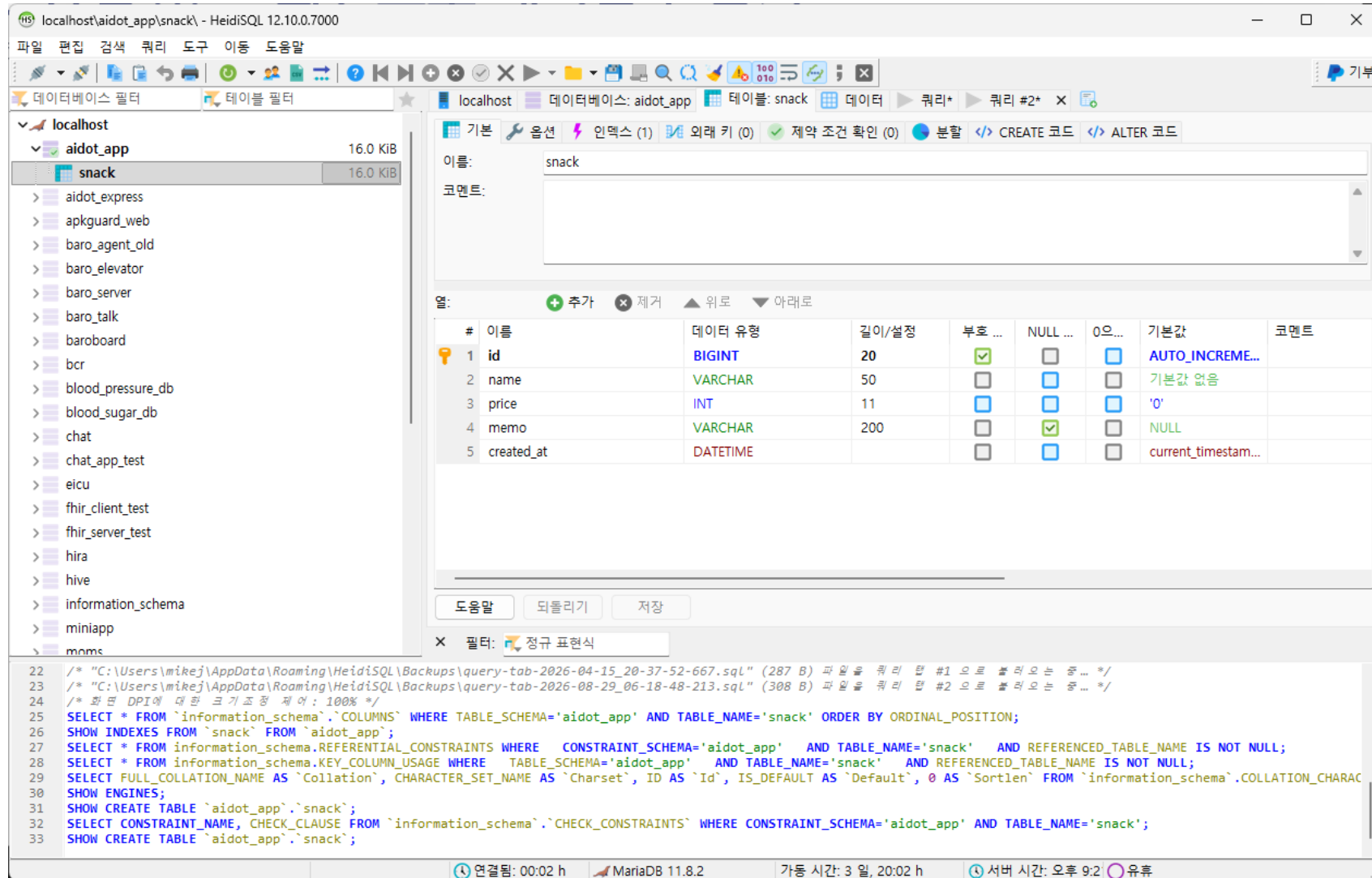
aidot_app 을 오른쪽 클릭
→ [새로 만들기 ▶ 테이블]

순서

누를 순서 : DB 오른쪽 클릭 → [새로 만들기 ▶ 테이블] → 이름 snack → 칼럼 5 개 추가 → [저장]

0 단계 다른 방법 ① — 화면으로 테이블 만들기

SQL 을 적는 게 어렵다면 , HeidiSQL 화면에서 마우스로 만들 수 있어요



localhost\aidot_app\snack - HeidiSQL 12.10.0.7000

파일 편집 검색 쿼리 도구 이동 도움말

데이터베이스 필터 테이블 필터

localhost 데이터베이스: aidot_app 테이블: snack 데이터 쿼리 #2*

기본 옵션 인덱스 (1) 외래 키 (0) 제약 조건 확인 (0) 분할 CREATE 코드 ALTER 코드

이름: snack

코멘트:

열: + 추가 - 제거 ▲ 위로 ▼ 아래로

#	이름	데이터 유형	길이/설정	부호 ...	NULL ...	0으...	기본값	코멘트
1	id	BIGINT	20	☒	☐	☐	AUTO_INCREME...	
2	name	VARCHAR	50	☐	☐	☐	기본값 없음	
3	price	INT	11	☐	☐	☐	'0'	
4	memo	VARCHAR	200	☐	☒	☐	NULL	
5	created_at	DATETIME		☐	☐	☐	current_timestam...	

도움말 되돌리기 저장

X 필터: 정규 표현식

```
22 /* "C:\Users\mikej\AppData\Roaming\HeidiSQL\Backups\query-tab-2026-04-15_20-37-52-667.sql" (287 B) 파일을 쿼리 탭 #1 으로 불러오는 중 ... */
23 /* "C:\Users\mikej\AppData\Roaming\HeidiSQL\Backups\query-tab-2026-08-29_06-18-48-213.sql" (308 B) 파일을 쿼리 탭 #2 으로 불러오는 중 ... */
24 /* 화면 DPI에 대한 크기 조정 제어: 100% */
25 SELECT * FROM `information_schema`.`COLUMNS` WHERE TABLE_SCHEMA='aidot_app' AND TABLE_NAME='snack' ORDER BY ORDINAL_POSITION;
26 SHOW INDEXES FROM `snack` FROM `aidot_app`;
27 SELECT * FROM information_schema.REFERENTIAL_CONSTRAINTS WHERE CONSTRAINT_SCHEMA='aidot_app' AND TABLE_NAME='snack' AND REFERENCED_TABLE_NAME IS NOT NULL;
28 SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='aidot_app' AND TABLE_NAME='snack' AND REFERENCED_TABLE_NAME IS NOT NULL;
29 SELECT FULL_COLLATION_NAME AS `Collation`, CHARACTER_SET_NAME AS `Charset`, ID AS `Id`, IS_DEFAULT AS `Default`, 0 AS `Sortlen` FROM `information_schema`.COLLATION_CHARAC
30 SHOW ENGINES;
31 SHOW CREATE TABLE `aidot_app`.`snack`;
32 SELECT CONSTRAINT_NAME, CHECK_CLAUSE FROM `information_schema`.`CHECK_CONSTRAINTS` WHERE CONSTRAINT_SCHEMA='aidot_app' AND TABLE_NAME='snack';
33 SHOW CREATE TABLE `aidot_app`.`snack`;
```

연결됨: 00:02 h MariaDB 11.8.2 가동 시간: 3 일, 20:02 h 서버 시간: 오후 9:2 유틸

② 이름에 **snack** 을 적고
[+ 추가] 로 칸을 5 개 만듭니다.
왼쪽 테이블 칼럼 내용 그대로 !

③ 오른쪽 위 [저장]
(또는 Ctrl+S)
왼쪽 목록에 **snack** 이 생기면
성공 !

0 단계 — 간식 데이터 추가하기

HeidiSQL 에서 딱 한 번만 하면 됩니다

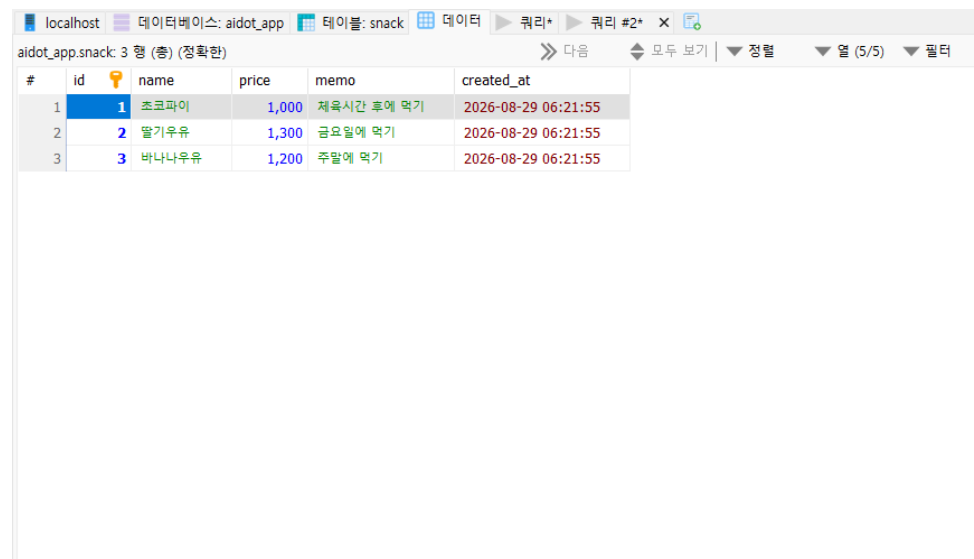
1 HeidiSQL 을 열고 위쪽 [쿼리] 탭을 누릅니다

2 아래 글자를 그대로 복사해서 붙여넣습니다

```
INSERT INTO `snack` (`name`, `price`, `memo`) VALUES (' 초코파이 ', 1000, '
체육시간 후에 먹기 ');
INSERT INTO `snack` (`name`, `price`, `memo`) VALUES (' 딸기우유 ', 1300, '
금요일에 먹기 ');
INSERT INTO `snack` (`name`, `price`, `memo`) VALUES (' 바나나우유 ', 1200, '
주말에 먹기 ');
```

3 실행 아이콘을 누릅니다 → 아래에 1 줄 성공한 것으로 나오면 됩니다
그 다음 , ‘ 데이터 ’ 탭을 열면 데이터가 보입니다

이런 데이터가 만들어져요



The screenshot shows the HeidiSQL interface with the 'snack' table selected. The table has 3 rows of data. The columns are: #, id, name, price, memo, and created_at. The data is as follows:

#	id	name	price	memo	created_at
1	1	초코파이	1,000	체육시간 후에 먹기	2026-08-29 06:21:55
2	2	딸기우유	1,300	금요일에 먹기	2026-08-29 06:21:55
3	3	바나나우유	1,200	주말에 먹기	2026-08-29 06:21:55

SQLite 를 쓰고 있다면 ?

같은 SQL 문을 SQLite 도구 (DB Browser for SQLite) 에 붙여넣고 실행합니다

0 단계 다른 방법 ② — 화면으로 간식 넣기

엑셀에 값을 적듯이 , [데이터] 탭에서 칸을 채우면 그대로 저장됩니다

HeidiSQL — aidot_app.snack

테이블

데이터

쿼리

+ 행 삽입

행 삭제

저장

F5

id	name	price	memo	created_at
1	초코파이	500	제일 인기	2026-08-24 09:03
2	바나나우유	1200	체육시간 뒤에	2026-08-24 09:12
(자동)	딸기우유	1300	금요일에 먹기	(자동)

id 와 created_at 은 비워 두세요 — 저장하면 서버가 알아서 채워 줍니다 .

저장하면 아래에 이렇게 나와요

```
/* 영향 받은 행 : 1 */ INSERT INTO `snack` (`name`, `price`, `memo`) VALUES (' 딸기우유 ', 1300, ' 금요일에 먹기 ');
```

① 위쪽 [데이터] 탭 클릭

② [+ 행 삽입] 을 누르면 맨 아래에 빈 줄이 생겨요 .

③ 칸을 눌러 값을 적고 Tab 키로 옆 칸 이동

④ 다 적었으면 [저장] (또는 Ctrl+S)

F5 를 누르면 다시 읽어와요 .

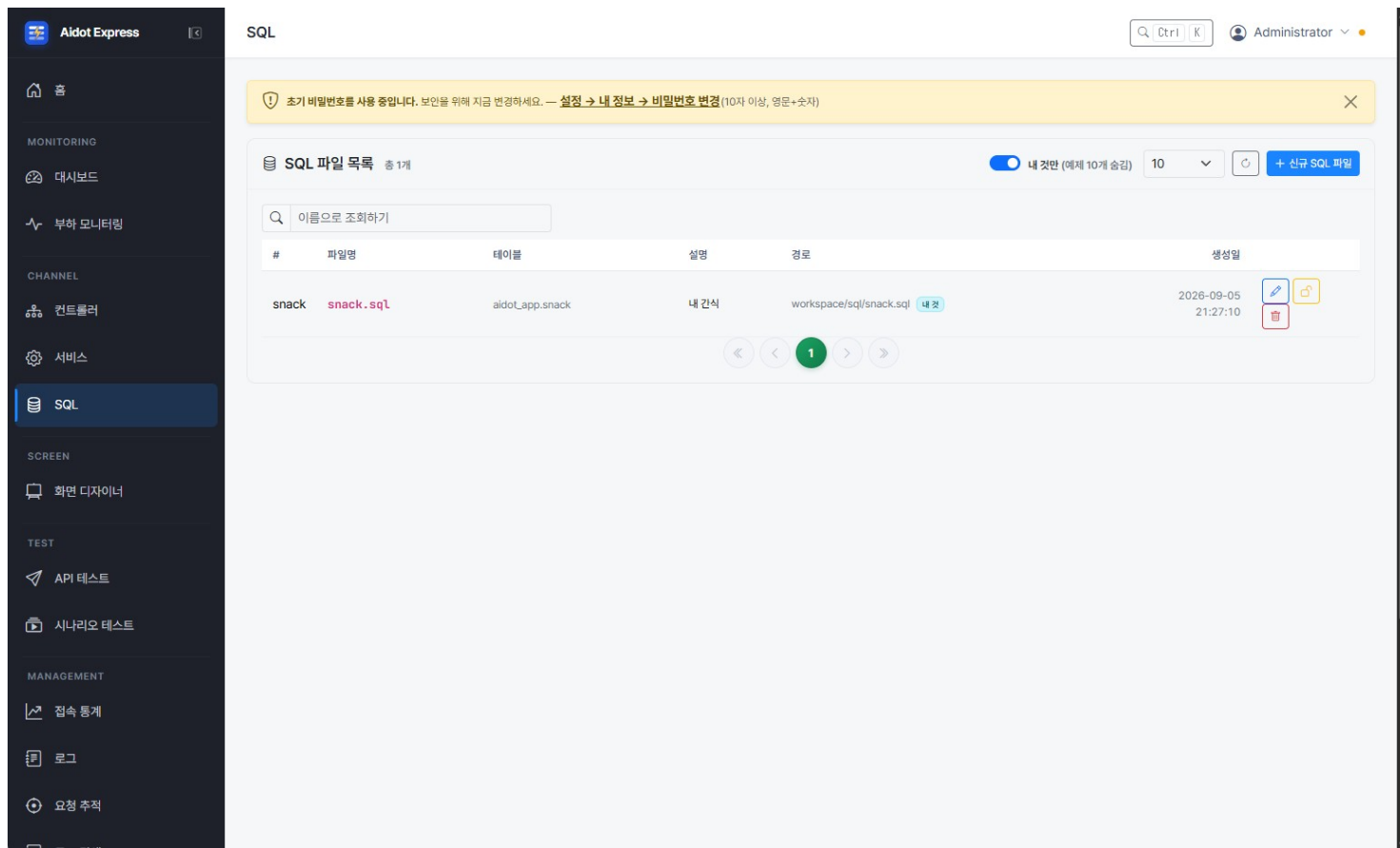
id 가 1, 2, 3... 으로 붙었는지 확인 !

순서

누를 순서 : [데이터] 탭 → [+ 행 삽입] → 칸 채우기 (Tab 이동) → [저장] → F5 로 확인

1 단계 — SQL 파일 만들기 ①

왼쪽 메뉴에서 [SQL] 을 누르면 목록이 나옵니다 . 오른쪽 위 파란 [+ 신규 SQL 파일] 을 누르세요



처음에는 목록이 비어 있어요 .

오른쪽 위 파란 버튼이 시작점입니다 .

[내 것만] 이 켜져 있으면

내가 만든 것만 보여요 .

예제까지 보려면 끄세요 .

만들고 나면 이 목록에

한 줄이 생깁니다 .

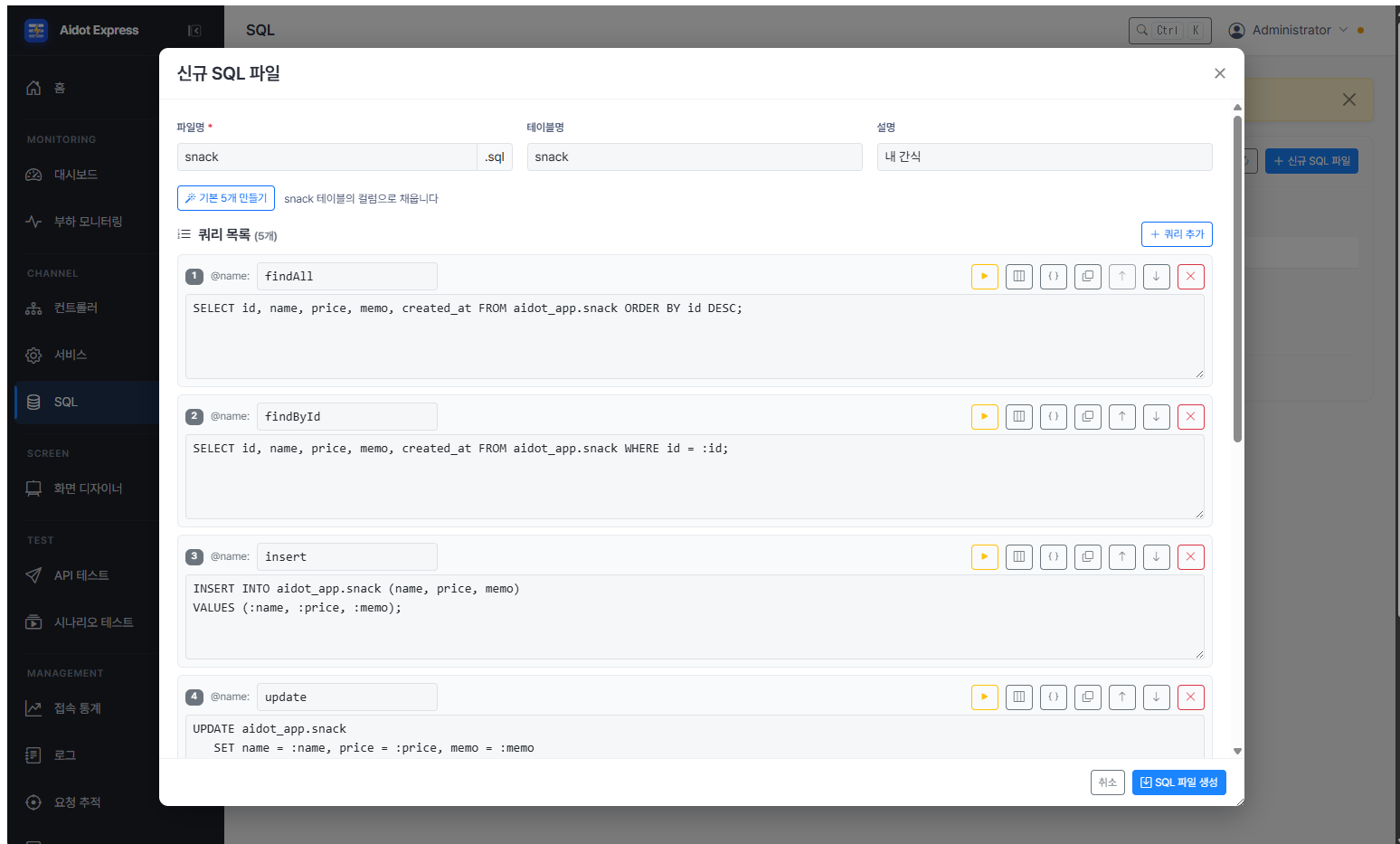
(경로에 workspace/sql/... 이라고 보여요)

순서

누를 순서 : 왼쪽 [SQL] → 오른쪽 위 [+ 신규 SQL 파일] 클릭

1 단계 — SQL 파일 만들기 ②

파일명 · 테이블명을 적고 [기본 5 개 만들기] 를 누르면 쿼리 5 개가 한 번에 만들어집니다



파일명은 **snack**
(.sql 은 저절로 붙어요)

테이블명을 적어야
컬럼까지 채워 줍니다 .
스키마를 정해 두었다면
aidot_app.snack 으로 나와요 .

[기본 5 개 만들기] 를 누르면
아래 쿼리 목록에 5 개가 생깁니다 .
(findAll · findById · insert ·
update · deleteById)

입력

적을 것 : 파일명 snack · 테이블명 snack → [기본 5 개 만들기] → [SQL 파일 생성] 클릭

1 단계 — SQL 파일 만들기 ③

[기본 5 개 만들기] 를 누르면 아래 5 개가 한 번에 만들어집니다 — 코드를 직접 입력할 필요 없어요

```
-- @name: findAll
SELECT id, name, price, memo, created_at
FROM aidot_app.snack
ORDER BY id DESC;

-- @name: findById
SELECT id, name, price, memo, created_at FROM aidot_app.snack WHERE id = :id;

-- @name: insert
INSERT INTO aidot_app.snack (name, price, memo)
VALUES (:name, :price, :memo);

-- @name: update
UPDATE aidot_app.snack
  SET name = :name, price = :price, memo = :memo
  WHERE id = :id;

-- @name: deleteById
DELETE FROM aidot_app.snack WHERE id = :id;
```

두 가지만 알면 돼요

-- @name: findAll

쿼리마다 붙이는 " 이름표 " 예요 .

서비스가 이 이름을 부르면
바로 아래 SQL 이 실행됩니다 .

:id :name :price

나중에 값이 들어올 " 빈 칸 " 이에요 .

클라이언트 (웹 화면) 에서 보낸 값이 이 자리에
안전하게 채워집니다 .

이름표 5 개는 정해진 이름이에요 .

다음 단계에서 그대로 골라 씁니다 .

다음

만들어진 뒤에는 꼭 [테스트 실행] → [수정 저장] 순서로 눌러야 다음 단계에서 고를 수 있어요 .

1 단계 — SQL 파일 만들기 ④

쿼리 카드 오른쪽의 ▶ 아이콘을 누르면 바로 아래에 결과가 펼쳐집니다

The screenshot shows the Aidot Express SQL editor interface. A modal window titled "SQL 파일 수정" (Edit SQL File) is open, displaying a query for the "snack" table. The query is: `SELECT id, name, price, memo, created_at FROM aidot_app.snack ORDER BY id DESC;`. The execution result is shown in a table format with 5 columns: id, name, price, memo, and created_at. The results are ordered by id in descending order.

id	name	price	memo	created_at
3	바나나우유	1200	주말에 먹기	2026-08-28T21:21:55.000Z
2	딸기우유	1300	금요일에 먹기	2026-08-28T21:21:55.000Z
1	초코파이	1000	제육시간 후에 먹기	2026-08-28T21:21:55.000Z

① 확인할 쿼리 카드를 찾고

② 오른쪽 노란 ▶ 아이콘을 누르세요

(옆 아이콘은 컬럼 · 변수 · 복사 · 순서 · 삭제)

결과가 그 카드 아래에 펼쳐져요 .

데이터가 테이블 모양으로 보이면 성공 !

빨간 글씨가 나오면 오타를 확인하세요 .

조회된 결과를 확인하세요 .

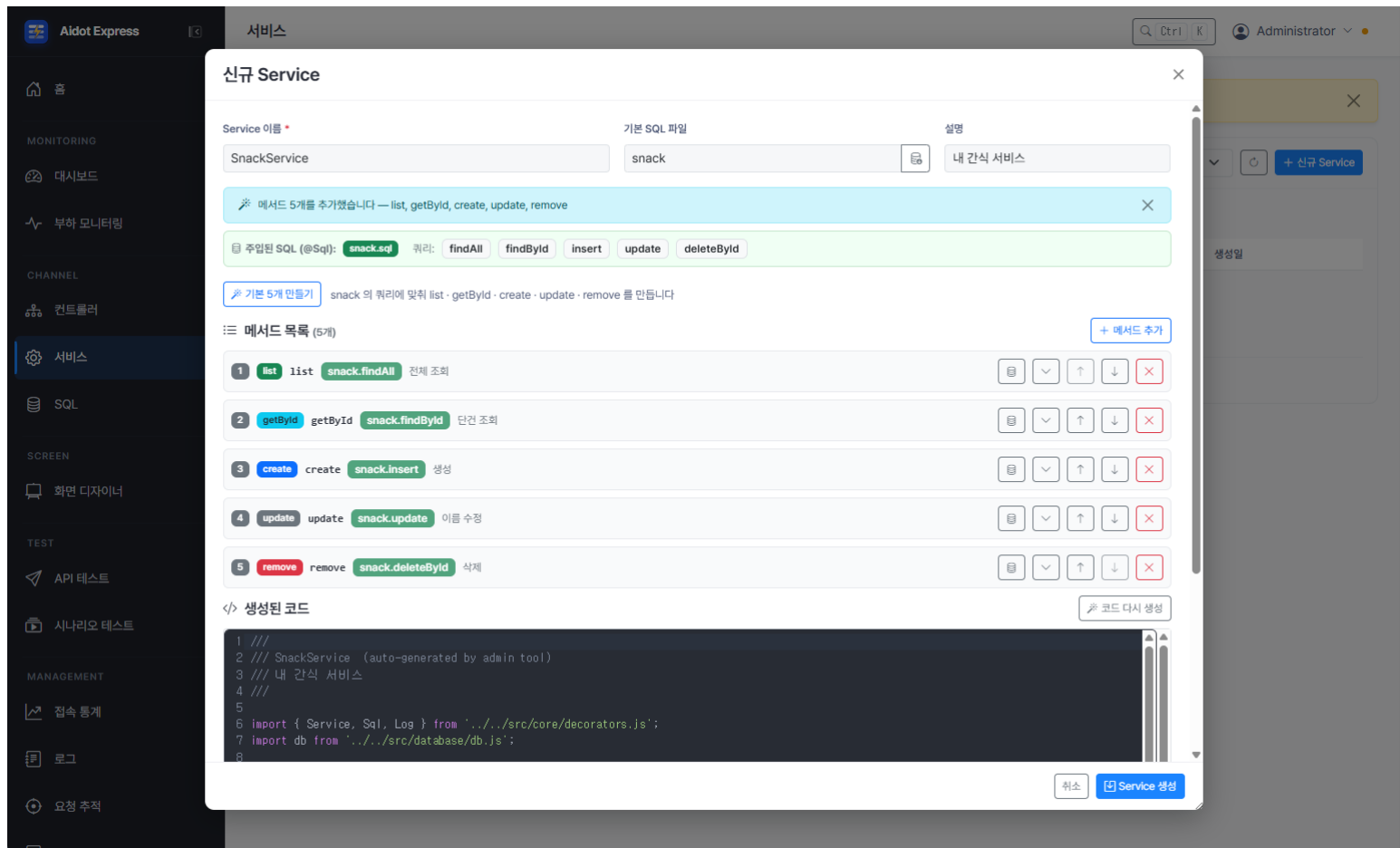
실행한 SQL 도 함께 보여 줍니다 .

순서

누를 순서 : 쿼리 카드의 ▶ 아이콘 → 아래 [실행] 클릭 → 조회된 결과 확인

2 단계 — 서비스 만들기

이름을 적고 기본 SQL 파일을 고른 뒤 [기본 5 개 만들기] 를 누르면 메서드가 생깁니다



이름은 **SnackService**

(맨 앞은 대문자 , 띄어쓰기 없이)

메서드 : 하나씩 실행할 수 있는 것

기본 SQL 파일에 **snack** 을 고르면
그 파일의 쿼리가 초록 배지로 보여요 .

[기본 5 개 만들기] 를 누르면
메서드가 생깁니다 .
list · getById · create ·
update · remove 5 개 .

입력

적을 것 : 이름 **SnackService** · 기본 SQL 파일 **snack** → [기본 5 개 만들기] → [**Service 생성**] 클릭

3 단계 — 컨트롤러 만들기 ①

이름과 기본 요청 경로를 적고 기본 Service 를 고릅니다

The screenshot shows the 'Aldot Express' interface with a '컨트롤러' (Controller) management window. The '신규 컨트롤러' (New Controller) dialog is open, displaying the following fields:

- 컨트롤러 이름 (Controller Name): SnackController
- 유형 (Type): DB
- 기본 요청 경로 (Basic Request Path): /api/snack
- 기본 Service (Basic Service): SnackService
- 설명 (Description): 내 간식 API
- 실시간 알림 (SSE) (Real-time Alert (SSE)): OFF
- 기본 5개 만들기 (Basic 5 Methods): list, getById, create, updateName, remove
- 라우트 목록 (Routes List): 라우트가 없습니다. 위의 '5개 기본 라우트 자동 생성'으로 한 번에 만들거나, 오른쪽 [라우트 추가] 로 하나씩 만드세요.
- 생성된 코드 (Generated Code): A dark area for the generated code.

이름은 **SnackController**

라우트 : 클라이언트 (웹 화면) 에서
하나씩 부를 수 있는 것

기본 요청 경로가 곧 주소예요 .
/api/snack →
localhost:7901/api/snack/

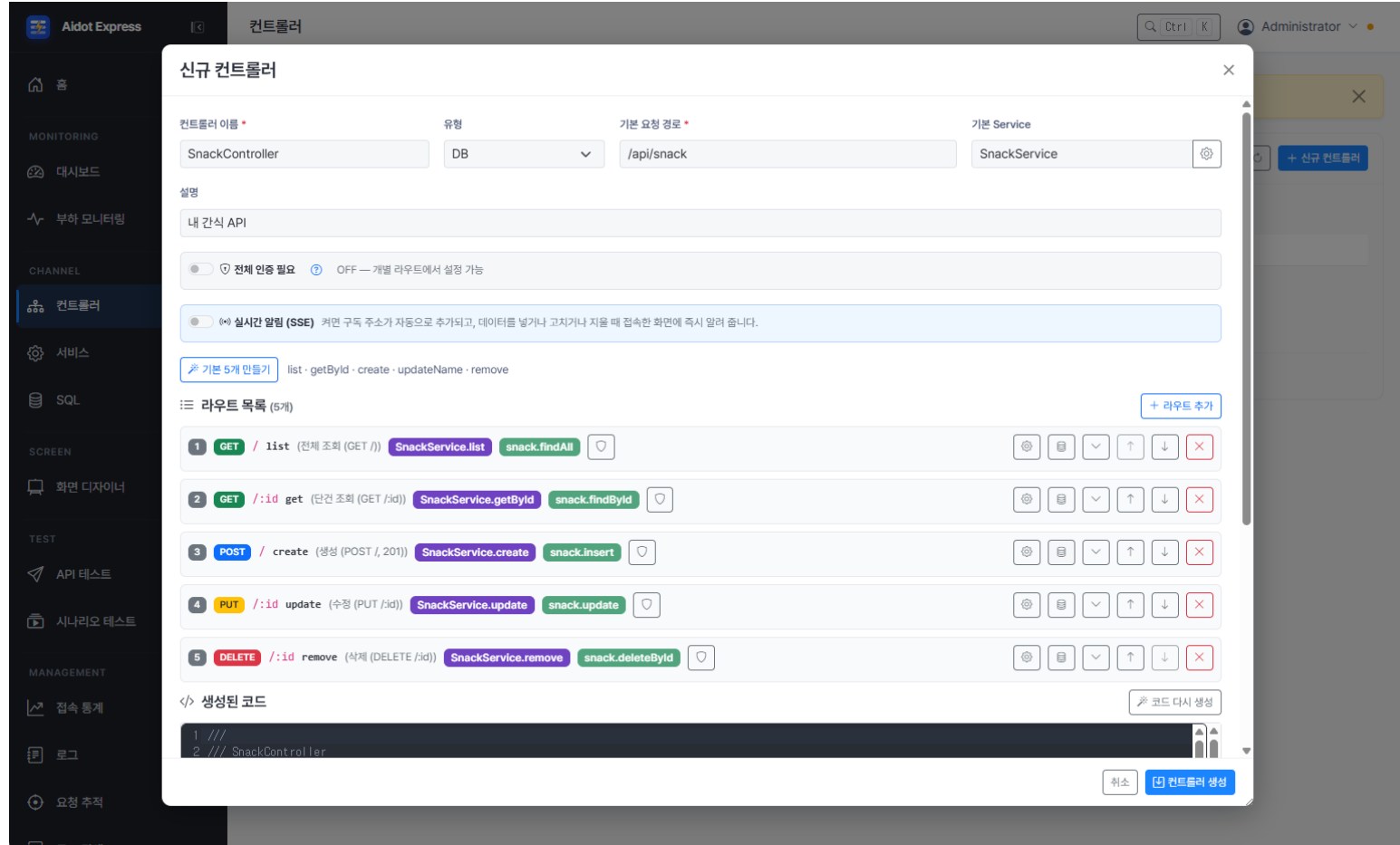
기본 Service 에 SnackService 를
입력합니다 .

입력

적을 것 : 이름 - **SnackController** · 경로 - **/api/snack** · 기본 Service - **SnackService**

3 단계 — 컨트롤러 만들기 ②

[기본 5 개 만들기] 한 번이면 라우트 5 개와 코드가 함께 만들어집니다



[기본 5 개 만들기] 를 누르면
라우트 5 개가 생기고 ,
아래 [생성된 코드] 까지
한 번에 만들어집니다 .

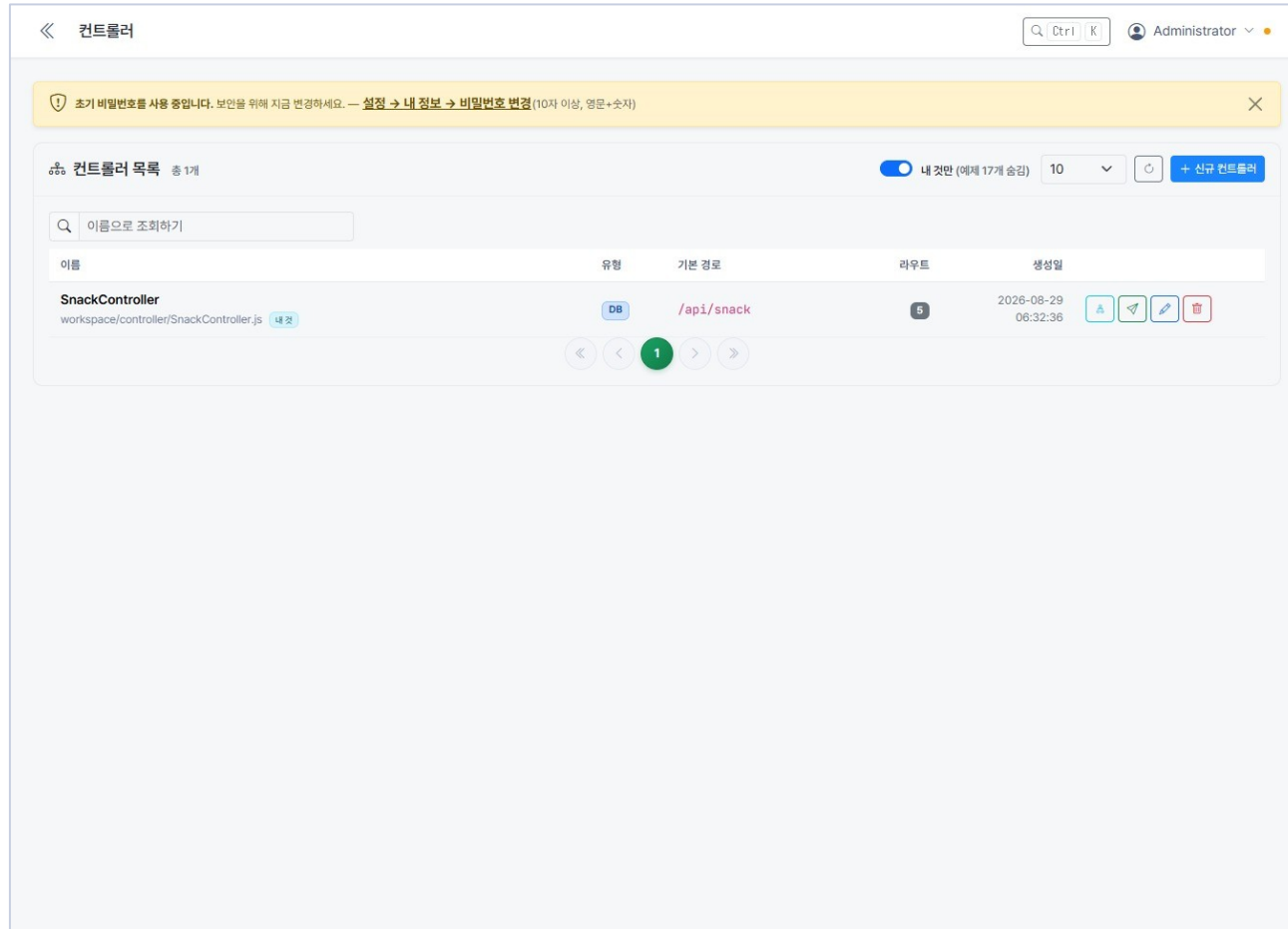
각 줄의 초록 · 보라 배지는
" 이 라우트가 어떤 서비스 · SQL 을
부르는지 " 를 보여 줘요 .

/:id 가 붙은 줄은
주소에 번호가 들어옵니다 .
/api/snack/2 → id 는 2

순서 누를 순서 : [기본 5 개 만들기] → 라우트 5 개와 코드 확인 → [컨트롤러 생성]

3 단계 — 잘 만들어졌는지 확인 ③

목록에 새 줄이 생기고 라우트 수가 보이면 바로 API 가 동작합니다



서버를 켜다 켜지 않아도
바로 동작해요 .

자물쇠 아이콘으로
이 컨트롤러를 잠깐 막을 수도
있어요 (파일은 지워지지 않아요).

4 단계 — 브라우저로 바로 확인하기

주소창에 그대로 쳐 보세요 . GET 방식으로 요청할 때는 브라우저로도 확인할 수 있어요



꼭 확인할 두 가지

- ✓ **code 가 200 인가요 ?**
200 이면 " 잘 됐어요 " 라는 뜻입니다 .
- ✓ **data 안에 간식이 보이나요 ?**
테이블에 넣어 둔 간식이 그대로 나오면 성공 !

글자가 다닥다닥 붙어 보이면 ?
브라우저 확장 "JSON Viewer" 를 깔면
예쁘게 줄이 맞춰져 보여요 .

POST · PUT · DELETE 는 브라우저에서 주소창에
입력하는 것만으로는 테스트를 못 해요 → 다음 페이지로
넘어가세요 .

5 단계 — 콘솔 [API 테스트] 로 새 간식 넣기

프로그램을 따로 깔지 않고 콘솔 안에서 바로 POST 방식으로 요청해 볼 수 있어요

aidot-express 관리자 콘솔

admin

홈

대시보드

컨트롤러

서비스

SQL

API 테스트

로그

사용자

POST ▾

/api/snack/

보내기

Body (보낼 내용)

```
{  
  "name": " 바나나우유 ",  "price": 1200,  "memo": " 체육시간 뒤에 "  
}
```

응답

201

```
{ "code": 201, "message": "Created",  
  "data": { "insertId": 2, "rowsAffected": 1 } }
```

- ① 메서드를 **POST** 로
- ② 주소는 **/api/snack/**
- ③ **Body** 에 내용 입력
- ④ **[보내기]**

insertId: 2 는
"2 번으로 저장했어요 "
rowsAffected: 1 은
" 한 줄 넣었어요 " 라는 뜻이에요 .

이제 브라우저에서
/api/snack/ 주소가 입력된 상태에서
새로고침하면 방금 넣은 간식이 보여요 !

순서

누를 순서 : [API 테스트] → POST 선택 → /api/snack/ → Body 입력 → [보내기] → 201 확인

6 단계 — 포스트맨으로 테스트하기

포스트맨은 웹 방식의 요청과 응답을 테스트할 때 가장 많이 사용하는 것 중 하나예요 . (postman.com 에서 내려받아 설치 → [New] → [HTTP Request])

Postman 새 간식 넣기

POST ▾

http://localhost:7901/api/snack/

Send

ParamsHeadersBodyraw ▾JSON ▾

```
{
  "name": " 딸기우유 ",  "price": 1300,  "memo": " 금요일에 먹기 "
}
```

Response201 Created · 34 ms

```
{ "code": 201, "message": "Created",
  "data": { "insertId": 3, "rowsAffected": 1 } }
```

이렇게 5 번 눌러 보세요

메서드	주소	Body 필요 ?
GET	/api/snack/	없음
GET	/api/snack/1	없음
POST	/api/snack/	있음 (이름 · 값)
PUT	/api/snack/1	있음 (고칠 내용)
DELETE	/api/snack/1	없음

Body 를 쓸 때는 꼭

[Body] → [raw] → [JSON]

을 골라 주세요 .

명령어 입력을 좋아한다면 curl 도 있어요 .

아래 한 줄이면 끝 !

```
curl localhost:7901/api/snack/
```

파트 2 : " 손으로 직접 " 만들어보기

콘솔이 대신 해 주던 일을 , 파일 3 개를 만들어서 똑같이 해 봅니다 .

코드 안을 들여다보면 콘솔이 무슨 일을 했는지 알게 돼요 .

1

workspace/sql/supply.sql

SQL 파일 — 저장소에서 꺼내는 방법

2

workspace/service/SupplyService.js

서비스 — SQL 을 실행하는 심부름꾼

3

workspace/controller/SupplyController.js

컨트롤러 — 요청을 받는 문지기

이름 정리 콘솔로 만들기 = **snack(간식)** · 파일로 만들기 = **supply(준비물)** · 서버 기본 예제 = **guestbook(방명록)**

직접 만들기 ① — 테이블 만들기

HeidiSQL 을 이용해 테이블을 만들고 데이터를 추가합니다 .

```
CREATE TABLE supply (  
  id      BIGINT AUTO_INCREMENT,  
  name    VARCHAR(50) NOT NULL,  
  count   INT DEFAULT 1,  
  memo    VARCHAR(200),  
  PRIMARY KEY (id));
```

```
INSERT INTO aidot_app.supply (name, count, memo) VALUES  
( '연필 ',      12, '2B'),  
( '공책 ',      5, '줄공책'),  
( '지우개 ',    3, NULL),  
( '색연필 ',   24, '12 색 두 통'),  
( '가위 ',      2, '안전가위'),  
( '폴 ',        4, '딱폴'),  
( '자 ',        1, '30cm'),  
( '물감 ',      1, '12 색'),  
( '스케치북 ',  2, '8 절'),  
( '필통 ',      1, NULL);
```

직접 만들기 ① — SQL 파일

workspace/sql/ 폴더에 supply.sql 파일을 새로 만드세요

```
-- 우리 반 준비물 목록

-- @name: findAll
SELECT id, name, count, memo FROM aidot_app.supply ORDER BY id DESC;

-- @name: findById
SELECT id, name, count, memo FROM aidot_app.supply WHERE id = :id;

-- @name: insert
INSERT INTO aidot_app.supply (name, count, memo)
VALUES (:name, :count, :memo);

-- @name: update
UPDATE aidot_app.supply SET name = :name, count = :count, memo = :memo
WHERE id = :id;

-- @name: deleteById
DELETE FROM aidot_app.supply WHERE id = :id;
```

VS Code 를 열고 파일을 만듭니다

1 이름표를 먼저 쓴다
-- @name: 뒤에 쿼리 이름을 적어요 .

2 문장 끝에 세미콜론
SQL 한 문장이 끝나면 ; 을 붙여요 .

3 값이 들어올 자리는 : 이름
:id, :name 처럼 콜론을 붙여요 .

supply 테이블도 만들어야 해요

HeidiSQL 에서 아래를 먼저 실행하세요 (0 단계와 같은 방법)

```
CREATE TABLE supply (
  id    BIGINT AUTO_INCREMENT,
  name  VARCHAR(50) NOT NULL,
  count INT DEFAULT 1,
  memo  VARCHAR(200),
  PRIMARY KEY (id));
```

이름

간식 (snack) 은 콘솔로 , 준비물 (supply) 은 파일로 — 주제가 달라서 이름이 겹치지 않습니다 .

직접 만들기 ② — 서비스 파일

workspace/service/ 폴더에 SupplyService.js 를 만드세요

```
import { Service, Sql, Log } from '../core/decorators.js';
import db from '../database/db.js';

@Service('SupplyService') // 이 이름으로 불러 쓸 수 있어요
export default class SupplyService {

  @Sql('supply') supplySql; // supply.sql 파일을 가져다 씁니다
  @Log log; // 로그를 출력할 때 사용할 수 있게 합니다

  async list() { // 전체 보기
    const sql = this.supplySql.get('findAll'); // SQL 이름표로 꺼내기
    const res = await db.execute(sql, {}); // SQL 실행하기
    return res.rows; // 컨트롤러 쪽으로 넘겨주기
  }

  async getById(id) { // 하나만 보기
    const sql = this.supplySql.get('findById');
    const res = await db.execute(sql, { id }); // :id 자리에 값 넣기
    return res.rows[0] ?? null;
  }
}
```

주요 코드는 3 가지예요

@Service

" 나는 심부름꾼이에요 " 라고
서버에게 알려 주는 이름표

@Sql('supply')

supply.sql 파일을
통째로 가져다 붙여 줍니다

db.execute(sql, {...})

SQL 을 진짜로 실행하고
결과를 받아 옵니다

create / update / remove 도 같은 모양으로 이어서 쓰면
돼요 .

직접 만들기 ③ — 컨트롤러 파일

workspace/controller/ 폴더에 SupplyController.js 를 만들면 주소가 열립니다

```
import {
  Controller, Log, Autowired,
  GetMapping, PostMapping, DeleteMapping,
} from '../core/decorators.js';

@Controller('/api/supply') // ← 이 주소로 열립니다
export default class SupplyController {

  @Autowired('SupplyService') supplyService; // 심부름꾼 부르기
  @Log log;

  @GetMapping('/') // GET /api/supply/
  async list(params) {
    return this.supplyService.list();
  }

  @GetMapping('/:id') // GET /api/supply/1
  async get(params) {
    return this.supplyService.getById(params.id);
  }
}
```

이번에 테스트할 때는 주소가 **/api/snack** 가 아니라 **/api/supply** 입니다 .

저장한다고 바로 반영되지 않아요

- 새로 추가한 컨트롤러는 서버가 켜져 있는 상태에서 인식되도록 할 수도 있습니다 .
- 대시보드에서 서버를 재시작해도 됩니다 .
- 만약 전체를 다시 시작하고 싶다면 그냥 Ctrl+C 로 끄고 다시 npm start 명령어로 실행할

확인하기 수 있습니다 .

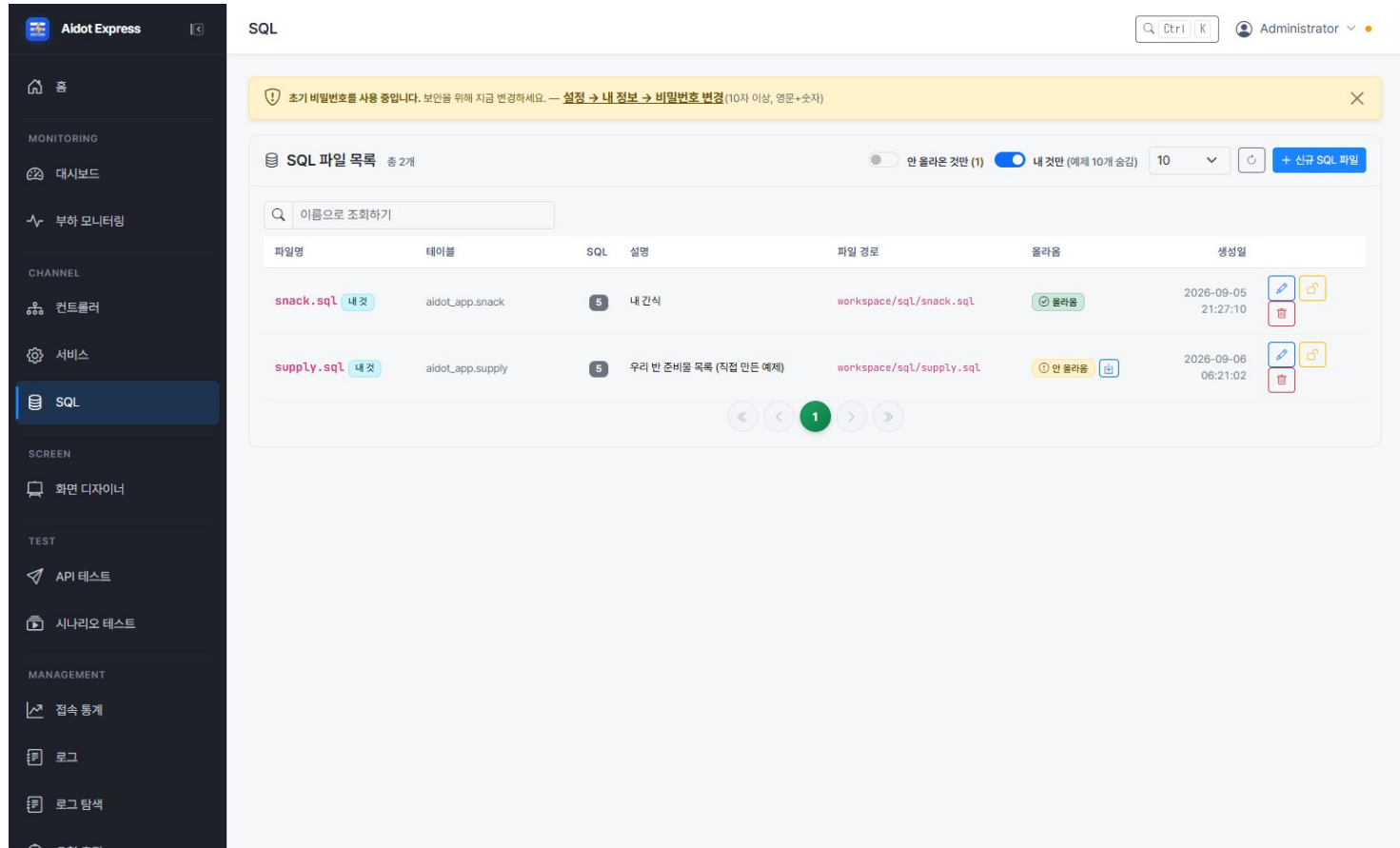
```
localhost:7901
  /api/supply/
```

콘솔 [컨트롤러] 목록에도 SupplyController 가 나타나요 .

콘솔로 만든 것과 손으로 만든 것 , 서버 입장에선 똑같아요 .

직접 만들기 — 콘솔에서 로딩하기

콘솔의 SQL, 서비스, 컨트롤러 각각의 목록에서 ‘안 올라옴’ 표시된 항목들을 로딩합니다



The screenshot shows the 'SQL' section of the Aidot Express console. A sidebar on the left contains navigation links for various system components. The main area displays a table of SQL files with columns for filename, table, SQL text, description, file path, loading status, and creation time. Two files are listed: 'snack.sql' and 'supply.sql'. The 'snack.sql' file has a green '올라옴' (Loaded) status, while 'supply.sql' has a yellow '안 올라옴' (Not Loaded) status. A search bar and pagination controls are also visible.

파일명	테이블	SQL	설명	파일 경로	올라옴	생성일
snack.sql	aidot_app.snack		내 간식	workspace/sql/snack.sql	올라옴	2026-09-05 21:27:10
supply.sql	aidot_app.supply		우리 반 준비물 목록 (직접 만든 예제)	workspace/sql/supply.sql	안 올라옴	2026-09-06 06:21:02

‘올라옴’ 칼럼의 로딩 상태

- 로딩된 것은 ‘올라옴’
로딩되지 않은 것은 ‘안 올라옴’ 표시

로딩하기

- [이것만 올리기] 아이콘을 누르면 그 항목만 로딩됩니다

7 단계 — 웹페이지에서 눌러 보기 (HTML)

서버에 이미 들어 있는 방명록 실습 화면이에요 . 주소만 치면 바로 열립니다

방명록 API 실습실

GET ① 글 전체 보기

전체 불러오기

GET ② 하나만 보기

불러오기

POST ③ 새 글 남기기

글 남기기

DELETE ⑤ 글 지우기

지우기

서버가 보낸 답

```
{
  "code": 200,
  "data": [
    {
      "id": 2,
      "writer":
        "홍길동",
      "message":
        "첫 방문!"
    }
  ]
}
```

주소창에 그대로 치세요

localhost:7901
/public/demo
/guestbook-lab.html

방명록 (guestbook) 은
서버에 원래 들어 있어요 .
내가 만든 /api/snack 으로
바꿔서 테스트해도 됩니다 .

버튼을 누를 때마다
오른쪽에 답이 바뀌고 ,
걸린 시간 (ms) 도 나와요 .

순서 ① 주소창에 페이지 주소 입력 → ② 버튼 누르기 → ③ 오른쪽 [서버가 보낸 답] 확인

7 단계 ① — VS Code 로 내 웹페이지 만들기

빈 파일 하나에 붙여넣기만 하면 됩니다 — 간식 목록을 불러옵니다

- ① VS Code 를 열고 [파일] → [폴더 열기]
아무 폴더나 됩니다 (예 : C:\work\snack-web)
- ② [파일] → [새 파일] → snack.html 로 저장
- ③ 오른쪽 코드를 통째로 붙여넣고 저장 (Ctrl+S)
- ④ 파일을 더블클릭해서 브라우저로 엽니다

⚠ 그냥 열면 CORS 오류가 납니다.
화면 (내 파일) 과 서버 (7901) 의 주소가 달라서
브라우저가 " 다른 집에서 온 요청 " 이라며 막아요 .

해결 — 서버 폴더에 넣고 서버 주소로 엽니다:

aidot-express/public/demo/snack.html 에 저장
→ http://localhost:7901/public/demo/snack.html

같은 주소에서 부르는 셈이 맞지 않습니다.

세 가지만 기억하세요

1

axios 를 불러온다

<script> 한 줄이면 준비 끝 .
설치할 것이 없습니다 .

2

axios.get(주소)

" 이 주소로 다녀와 " 라는 심부름 .
res.data 에 답이 들어옵니다 .

3

await 와 async

" 답이 올 때까지 기다려 " 라는 뜻 .

fetch 와 무엇이 다른가요 ?

axios 는 JSON 을 알아서 풀어 줍니다 .

fetch 는 res.json() 을 한 번 더 불러야 해요 .

파일

다음 장에 붙여넣을 코드 전체가 있습니다

7 단계 ② — 붙여넣을 코드 (전체)

snack.html — 아래를 그대로 복사해 붙여넣으세요

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="utf-8" />
  <title>간식 목록</title>
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>
<body style="font-family: sans-serif; padding: 24px">

  <h1>간식 목록</h1>
  <button onclick="loadSnacks()">불러오기</button>
```

```
async function loadSnacks() {
  try {
    const res = await axios.get('/api/snack/');
    const list = res.data.data ?? res.data;
    document.getElementById('rows').innerHTML = list
      .map(s => '<tr><td>' + s.id + '</td><td>' + s.name +
        '</td><td>' + s.price + '</td></tr>')
      .join('');
  } catch (e) {
    alert('불러오지 못했습니다: ' + (e.response?.status ?? e.message));
  }
}
</script>
```

읽는 요령

1

맨 위 script 한 줄

설치 없이 axios 를 쓰게 해 줍니다 .
인터넷이 연결돼 있어야 해요 .

2

res.data.data ?? res.data

서버가 { code, data } 로 감싸 주면 안쪽을 ,
그냥 배열이면 그대로 씁니다 .

3

try / catch

실패해도 화면이 멈추지 않습니다 .
무엇이 잘못됐는지 알려 줘요 .

안 나오나요 ?

- 서버가 켜져 있는지 (npm start)
- 5 단계에서 /api/snack 을 만들었는지

파일

저장 : [aidot-express/public/demo/snack.html](https://aidot-express.com/public/demo/snack.html) → 열기 : localhost:7901/public/demo/snack.html

8 단계 ① — Vue 3 프로젝트 만들기

명령 세 줄이면 준비 끝 — TypeScript 는 쓰지 않습니다

1 프로젝트 만들기

명령 프롬프트

```
C:\work> npm create vue@latest snack-vue -- --router --pinia
```

명령 프롬프트

```
VITE v8.1.5 ready in 512 ms  
  
→ Local: http://localhost:5173/  
→ press h + enter to show help
```

2 묻지 않고 바로 만들기

-- --router --pinia

제대로 됐는지 보는 법

--typescript false

안 물어봄

src/main.js

쓰지 마세요

왜 두 개를 켜 두나요 ?

7901 = 서버 (자료 담당)
5173 = 화면 (보여주기 담당)

검은 창 2 개를 나란히 켜 둡니다 .

3 설치하고 axios 추가하기

명령 프롬프트

```
C:\work> cd snack-vue && npm install && npm install axios && npm run dev
```

npm install 이 실패하면 ?

Node 22 이상인지 확인하세요 .

포트가 쓰이면 vite 가 5174 로 올립니다 .

순서

순서 : npm create vue@latest snack-vue -- --router --pinia → npm install → npm install axios
→ npm run dev

8 단계 ② — 고칠 파일은 두 개뿐

파일 두 개만 고칩니다 — vite.config.js 와 views/HomeView.vue

폴더 구조 (만들어진 그대로)

snack-vue/	
package.json	axios 를 여기에 추가했습니다
src/	
main.js	시작점 — 그대로
vite.config.js	① 프록시를 넣습니다
node_modules/	npm install 이 만듭니다
stores/	건드리지 않습니다
components/	건드리지 않습니다
views/HomeView.vue	② 여기에 코드를 붙여넣습니다
App.vue · router/	만들어진 그대로

파일 두 개만 고치면 됩니다 . 나머지는 만들어진 그대로 두세요 .

고칠 파일 ① — vite.config.js

```
export default defineConfig({
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:7901',
        changeOrigin: true,
      },
    },
  },
  // plugins.resolve 는 그대로 두세요
})
```

이게 없으면 ?

화면 (5173) 과 서버 (7901) 주소가 달라서 브라우저가
" 다른 집에서 온 요청 " 이라며 막습니다 (CORS 오류).
프록시를 걸면 브라우저는 같은 주소로 부른 셈이 돼요 .

화면
localhost:5173



프록시
(vite)



서버
7901

8 단계 ③ — HomeView.vue (전체)

src/views/HomeView.vue — 그대로 붙여넣으세요 (axios 는 import 로 불러옵니다)

```
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

const snacks = ref([])
const error = ref('')
const loading = ref(false)

async function loadSnacks() {
  loading.value = true
  error.value = ''
  try {
    const res = await axios.get('/api/snack/')
    snacks.value = res.data.data ?? res.data
  } catch (e) {
    error.value = '불러오지 못했습니다: ' + (e.response?.status ?? e.message)
  } finally {
    loading.value = false
  }
}

onMounted(loadSnacks)
</script>
```

```
<template>
<main style="padding: 24px">
  <h1>간식 목록</h1>
  <button :disabled="loading" @click="loadSnacks">
    {{ loading ? '불러오는 중...' : '다시 불러오기' }}
  </button>
  <p v-if="error" style="color: crimson">{{ error }}</p>
  <table v-else border="1" cellpadding="6">
    <thead>
      <tr><th>번호</th><th>이름</th><th>가격</th></tr>
    </thead>
    <tbody>
      <tr v-for="s in snacks" :key="s.id">
        <td>{{ s.id }}</td>
        <td>{{ s.name }}</td>
        <td>{{ s.price }}</td>
      </tr>
    </tbody>
  </table>
</main>
</template>
```

읽는 요령

1

import axios from 'axios'

CDN 이 아니라 npm 으로 설치한 것을 script setup 안에서 불러옵니다.

2

ref() 와 onMounted()

ref 에 담으면 화면이 저절로 다시 그려지고, onMounted 로 열리자마자 한 번 불러옵니다.

3

try / catch / finally

실패해도 멈추지 않고, loading 도 반드시 꺼 줍니다.

안 나오나요 ?

- vite.config.js 에 프록시를 넣었는지
- 7901 과 5173 을 둘 다 켜는지

확인

확인 : 표에 간식이 나오면 성공 · 안 나오면 F12 → [네트워크] 에서 /api/snack 요청을 보세요

8 단계 — Vue 화면 확인하기 ③

http://localhost:5173 을 열면 위쪽에 메뉴 2 개가 있는 화면이 나옵니다

방명록 API 실습실 Vue 3 + Vite

방명록 5 가지 기능

실시간 알림 (SSE)

① 전체 보기

전체 불러오기

id	이름	한마디
2	홍길동	첫 번째 방문 기념 !
1	아이닷	방명록 예제입니다

③ 새 글 남기기

이름한마디

아이닷반갑습니다 !

글 남기기

```
{ "code": 201, "message": "Created",  
  "data": { "insertId": 3 } }
```

글을 남기거나 지우면 왼쪽 표가 저절로 다시 불러와집니다 — Pinia 스토어가 목록을 다시 읽기 때문이에요 .

HTML 페이지와 뭐가 다른가요 ?

- 화면과 자료 관리가 나뉨
- 표가 자동으로 새로고침
- 화면 (주소) 이 여러 개

오류가 나면 ?

F12 를 눌러 [콘솔] 탭의
빨간 글씨를 보세요 .
서버 오류는 [네트워크] 탭 .

완성한 화면을 배포하려면
npm run build → dist 폴더

순서

확인할 것 : 목록이 나오는가 · [넣기] 후 표가 자동으로 늘어나는가 · 응답 code 가 200/201 인가

9 단계 — 서버가 먼저 알려 주는 기능 (SSE)

지금까지는 " 물어보면 답하는 " 방식이었어요 . 이번엔 서버가 먼저 말을 겁니다

이전 방식 — 계속 물어보기

새 소식 있어요 ?

아직요 .

새 소식 있어요 ?

아직요 .

새 소식 있어요 ?

네 ! 하나 있어요 .

5 초마다 묻는다면 → 하루 17,280 번 . 서버도 힘들고 , 소식은 최대 5 초 늦게 옵니다 .

SSE — 한 번 연결해 두기

연결할게요 ! (딱 1 번)

소식이 생겼어요 ①

소식이 생겼어요 ②

소식이 생겼어요 ③

연결은 1 번뿐 . 소식이 생기는 즉시 (수십 ms) 도착합니다 .

	계속 물어보기 (폴링)	SSE	한마디로
소식이 오는 속도	묻는 주기만큼 늦음	즉시	SSE 가 빠름
서버 부담	요청이 계속 쌓임	연결 1 개	SSE 가 가벼움
끊겼을 때	다음 요청 때 복구	자동 재연결 + 놓친 것 재전송	브라우저가 알아서

9 단계 — 서버에 붙어 있는 SSE 주소 3 개

aidot-express 에 이미 들어 있어요 . 따로 만들 필요 없이 바로 쓰면 됩니다

GET

/api/events/stream?channel=guestbook

연결해서 소식 받기

POST

/api/events/publish

소식 보내기 (모두에게 전달)

GET

/api/events/stats

지금 몇 명이 붙어 있나 보기

서버 코드는 이게 전부예요

```
@SseMapping('/stream')                // ← 이 한 줄이 스트림을 만든다
async stream(params, req) {
  req.sse.send({ event: 'hello', data: { message: ' 연결됨 ' } });
  return params.channel || 'demo';    // 반환값 = 구독할 채널 이름
}

// 어디서 어떻게 소식을 보낼 수 있냐
sseHub.publish('guestbook', { message: ' 새 글이 올라왔어요 ' });
```

알아서 해 주는 것들



끊기면 자동 재연결

브라우저가 3 초 뒤 스스로 다시 붙어요 .



놓친 소식 다시 보내기

연결이 끊긴 사이의 것만 골라서 보내 줍니다 .



25 초마다 살아있나 확인

중간 장비가 연결을 끊지 못하게 신호를 보냅니다 .



채널로 나눠 보내기

channel 이름이 다르면 서로 안 섞여요 .

콘솔 [부하 모니터링] 화면도

이 기능으로 동작합니다 . · 설계 설명 :

[docs/SSE_DESIGN.md](#)

9 단계 — 콘솔에서 체크 하나로 실시간 켜기

코드를 안 써도 됩니다 . [컨트롤러] 편집 화면의 스위치 하나면 끝

aidot-express 관리자 콘솔

admin

홈

대시보드

컨트롤러

서비스

SQL

API 테스트

로그

사용자

실시간 알림 (SSE)

켜면 구독 주소가 자동으로 추가되고 , 넣거나 고치거나 지울 때 즉시 알려 줍니다 .

채널 이름

snacks

구독 주소

/api/snacks/events

연결해 보기

연결됨

14:31:02

change { "action": "created", "id": 7 }

라우트 목록

유형	메서드	경로	핸들러	하는 일
list	GET	/	list	전체 보기
create	POST	/	create	넣기 + 알림 발행
remove	DELETE	/:id	remove	지우기 + 알림 발행
sse	GET	/events	events	← 자동으로 추가됨

누르는 순서

- ① [실시간 알림] 스위치 켜기
- ② 채널 이름 확인 (자동 제안)
- ③ [컨트롤러 생성]
- ④ [연결해 보기] 로 확인

켜는 순간 코드에
@SseMapping 라우트와
sseHub.publish(...) 가
자동으로 들어갑니다 .

인증을 켜 컨트롤러라면
주소 뒤에
?access_token= 토큰
을 붙여서 구독합니다 .

순서

누를 순서 : [실시간 알림] 스위치 → 채널 확인 → [컨트롤러 생성] → [연결해 보기] → 다른 탭에서 POST 해 보기

10 단계 — 화면도 실시간으로 (화면 디자이너)

만든 API 로 화면까지 만들고 , 스위치 하나로 " 저절로 새로고침 " 되게 합니다

aidot-express 관리자 콘솔admin

홈
대시보드
컨트롤러
서비스
SQL
화면 디자이너
API 테스트
로그

데이터 소스 고르기

컨트롤러라우트

SnackControllerGET /api/snacks/

실시간 자동 갱신

이 API 에 실시간 스트림이 있습니다 . 켜 두면 데이터가 바뀔 때마다 화면이 스스로 목록을 다시 읽습니다 (새로고침 불필요).

구독 주소 : /api/snacks/events

만들어진 화면

간식 목록

● 실시간

갱신됨

id	name	price	memo
3	딸기우유	1300	← 방금 다른 사람이 추가한 것
2	바나나우유	1200	체육시간 뒤에

누르는 순서

- ① [화면 디자이너] → 화면 추가
- ② 목록 위젯의 데이터 소스 고르기
- ③ [실시간 자동 갱신] 켜기
- ④ 내보내기 → `npm run dev`

스트림이 없는 API 에는
이 스위치가 아예 안 보여요 .
(컨트롤러에서 먼저
[실시간 알림] 을 켜야 합니다)

창을 두 개 열어 두고
한쪽에서 간식을 넣어 보세요 .
다른 쪽 목록이 저절로 늘어납니다 .

순서

누를 순서 : 데이터 소스 선택 → [실시간 자동 갱신] 켜기 → 내보내기 → 두 창으로 확인

9 단계 — 창 두 개로 실시간 확인하기

같은 페이지를 두 개 열어 놓고 , 한쪽에서 보내면 다른 쪽에 바로 나타납니다

localhost:7901/public/demo/guestbook-lab.html

창 A — 보내는 쪽

채널 이름
guestbook

보낼 메시지
안녕하세요 !

연결

메시지 보내기

● 연결됨

14:31:02 보냄 → 안녕하세요 ! (받은 사람 2 명)
14:30:55 서버와 연결되었습니다 .

즉시 !
→

localhost:5173/events

창 B — 받는 쪽 (Vue 화면)

● 연결됨

14:31:02 id 7 안녕하세요 !
14:30:58 구독 시작 (channel=guestbook)
14:30:58 guestbook 채널에 연결되었습니다

① [연결] 을 누르면 ● 표시가 초록색으로 바뀐다

② 한쪽에서 보낸 글이 다른 쪽에 바로 나타난다

③ 서버를 껐다 켜면 " 재접속 중... " 후 저절로 다시 붙는다

보너스

방명록에 글을 남기면 서버가 저절로 알려 줘요 . 검은 창에서도 : `curl -N localhost:7901/api/events/stream?channel=guestbook`

11 단계 — 페이지 나눠 보기 (페이지네이션)

간식이 24 개가 되면 한 화면에 다 나오지 않아요 . 10 개씩 끊어서 보여 줍니다

```
-- 지금까지 만든 findAll
SELECT id, name, price, memo, created_at
FROM snack
ORDER BY id DESC;

-- 24 행이 한꺼번에 옵니다 .
-- 행이 수천 개가 되면 -- 화면도 느리고 서버도 힘듭니다 .
-- 페이지네이션은
--   "10 개씩만 주세요 "
-- 라고 요청하는 방법입니다 .
```

필요한 것 두 가지

page

몇 번째 페이지인지 .

1 부터 셉니다 .

perPage

한 페이지에 몇 개인지 .

보통 10 이나 20 을 씁니다 .

이 둘만 있으면
서버가 알아서 잘라 줍니다 .

다음

SQL 은 그대로 두고 실행하는 방법만 바꿉니다 — 다음 장에서 봅니다 .

11 단계 — 서버는 db.executeList 를 씁니다

SQL 을 새로 쓰지 않습니다 . 실행 함수만 바꾸면 LIMIT/OFFSET 을 알아서 붙여 줍니다

```
// 서비스 파일
async listPaged(opts) {
  const sql = this.snackSql.get('findAll');
  return await db.executeList(sql, {}, {
    page: opts.page,
    perPage: opts.perPage,
  });
}
// 돌려주는 모양
// rows   : 이번 페이지의 행 10 개
// header : total 24, page 1,
//         perPage 10, totalPages 3
```

핵심은 한 줄

db.execute → db.executeList

SQL 은 그대로 두고

실행 함수만 바꿉니다 .
LIMIT/OFFSET 은 알아서 붙입니다 .

rows 와 header

rows 는 이번 페이지의 행 ,

header 는 페이지를 그릴 때
쓰는 숫자들이에요 .

SQLite · MariaDB 모두
같은 함수로 됩니다 .

요령

SQL 파일은 손대지 않습니다 — findAll 을 그대로 씁니다 .

11 단계 — 컨트롤러에 주소 하나 더

[컨트롤러] 편집 → [라우트 추가] → 유형에서 listPaged 를 고르면 이 코드가 들어갑니다

```
// 컨트롤러 파일
@GetMapping('/paged')
async listPaged(params) {
  return this.snackService.listPaged({
    page:    Number(params.page)    || 1,
    perPage: Number(params.perPage) || 10,
  });
}
// 새 주소
// GET /api/snack/paged?page=1&perPage=10
```

왜 Number(...) 인가요

주소로 오는 값은 글자

"1" 을 숫자 1 로 바꿉니다 .

|| 1 은 빠졌을 때
1 페이지로 본다는 뜻이에요 .

기존 주소는 그대로

GET /api/snack/ 도 남겨 둡니다 .

전체가 필요한 곳이 있으니까요 .

유형을 고르면 컨트롤러와
서비스에 코드가 함께 들어갑니다 .

순서

[라우트 추가] → 유형 listPaged → [컨트롤러 수정] 으로 저장

11 단계 — 진짜 나눠는지 확인하기

[API 테스트] 에서 page 를 바꿔 가며 눌러 보세요 (간식 24 개 기준)

```
GET /api/snack/paged?page=1&perPage=10
```

```
→ rows 10 개 · totalPages 3
```

```
GET /api/snack/paged?page=2&perPage=10
```

```
→ rows 10 개
```

```
GET /api/snack/paged?page=3&perPage=10 → rows 4 개 (24 - 20 = 4)
```

```
-- 마지막 페이지는 남은 만큼만 옵니다 .
```

확인할 것 세 가지

숫자가 맞는가

- ① rows 개수 = perPage
- ② header.total = 전체 행수
- ③ 마지막은 남은 만큼

안 맞으면

SQL 의 ORDER BY 를 보세요 .

정렬이 없으면 페이지마다
순서가 뒤바뀝니다 .

여기까지 되면
화면에 붙일 준비 끝 .

확인

총 24 행 · 10 개씩 → 3 페이지 . 마지막 페이지만 4 행이면 정상입니다 .

11 단계 — 화면 디자이너에서 페이지 붙이기

[화면 디자이너] → 위젯에서 List (Paged) 를 놓고 방금 만든 주소에 연결합니다

누르는 순서

- ① 화면 디자이너 → 프로젝트 열기
- ② 왼쪽 위젯에서 List (Paged) 끌어다 놓기
- ③ 오른쪽 [데이터 소스] → [편집]
- ④ `SnackController` → 라우트 `listPaged`
- ⑤ `perPage` 확인 (기본 10) ⑥ [미리보기] 로 숫자 버튼 확인

List 와 무엇이 다른가

List

받은 행을 그냥 다 그립니다 .

페이지가 없습니다 .

List (Paged)

맨앞 · 이전 · 숫자 · 다음 · 맨뒤

페이지가 위젯 안에 있습니다 .
직접 만들 필요가 없어요 .

주소는 꼭 `listPaged` 로 .
보통 목록 주소는 `header` 를

요령

페이지는 `totalPages` 를 알아야 숫자 버튼을 그릴 수 있습니다 .

12 단계 — 파일 업로드 (먼저 방식부터)

기본은 파일을 글자로 바꿔 JSON 으로 받습니다 — multipart 도 됩니다 (뒤에서 봅니다)

브라우저

```
FileReader.readAsDataURL(file)
```

↓

```
"data:image/png;base64,iVBORw0..."
```

↓

```
{ fileName: "간식.png",  
  fileBase64: "iVBORw0..." }
```

↓

보통의 POST · JSON 서버

```
Buffer.from(fileBase64, "base64")
```

↓

```
public/uploads/ 에 저장
```

왜 이렇게 하나요

컨트롤러가 평소와 똑같음

params 하나만 받으면 됩니다 .

multipart 를 쓰면 이 컨트롤러만
설정이 달라져요 .

이미 쓰고 있어요

MCI 정의서 엑셀 ,

백업 zip 이
전부 이 방식입니다 .

⚠ 본문 상한 10MB.
base64 는 33% 커지므로

주의 더 큰 파일이 필요하면 .env 의 BODY_LIMIT 을 올리세요 .

12 단계 — 쉬운 길 : 이미 있는 업로드부터

새로 만들기 전에 , 콘솔에 이미 있는 업로드를 눌러 보면 방식이 눈에 들어옵니다

① MCI 컨트롤러 만들기

[MCI 채널] → [MCI 컨트롤러 생성]
인터페이스 정의서 (엑셀) 를 올리면
컨트롤러가 통째로 만들어집니다 .

② 백업 복원

[백업 / 복원] → zip 올리기 설정과 코드가 되 돌아옵니다 .
-- 둘 다 같은 방식입니다 .

눈으로 확인하기

F12 → 네트워크

요청 본문을 열어 보면

fileBase64 가 보입니다 .
글자로 바뀌어 간 것이죠 .

연습용 파일

docs 폴더의

mzd_deptmmt_l01.xls 를
MCI 마법사에 올려 보세요 .

먼저 이걸 해 보면
다음 장이 훨씬 쉽습니다 .

요령

남이 만든 것을 먼저 눌러 보는 편이 , 설명을 읽는 것보다 빠릅니다 .

12 단계 — 업로드 컨트롤러 직접 만들기

[신규 컨트롤러] 로 빈 컨트롤러를 만들고 아래를 붙여넣습니다

```
@PostMapping('/')
async upload(params) {
  const { fileName, fileBase64 } = params;

  // ① 이름을 그대로 믿지 않는다
  const safe = path.basename(fileName)
    .replace(/[^\w.\- 가 - 힉 ]/g, '_'); // ② 글자를 다시 파일로
  const raw = fileBase64.includes(',')
    ? fileBase64.split(',').pop() : fileBase64;
  const buf = Buffer.from(raw, 'base64');
  fs.writeFileSync(dir + '/' + safe, buf); return { data: { fileName: safe } };}
```

꼭 해야 하는 두 가지

① 파일 이름 걸러내기

".././etc/passwd" 같은 이름이 오면

프로젝트 밖에 씁니다 .

basename 으로 경로를 떼세요 .

② 덮어쓰지 않기

같은 이름이면 -2, -3 을 붙입니다 .

덮어쓰면 되돌릴 수 없어요 .

화면은 HTML 실습 페이지로 —
<public/demo/upload-lab.html> 참고 .

완성

올린 파일은 /uploads/ 파일명 주소로 바로 열립니다 .

12 단계 — 업로드 두 가지 방식 · 멀티파트

같은 파일을 두 가지 길로 보낼 수 있습니다 . 언제 어느 쪽을 쓰는지만 알면 됩니다

```
# ① base64 (JSON) — 지금까지 쓰던 방식
POST /api/uploads/
Content-Type: application/json
{ "fileName": "간식.png",
  "fileBase64": "iVBORw0..." }

# ② 멀티파트 — 브라우저 <form> 이 쓰는 방식
POST /api/uploads/multipart
Content-Type: multipart/form-data

curl -F "file=@간식.png" \
      -H "Authorization: Bearer <토큰>" \
      .../api/uploads/multipart

# 돌려주는 모양은 둘이 같습니다
# { fileName, size, url }
```

어느 쪽을 쓸까요

base64 를 쓸 때

이미 JSON 을 주고받는 화면에서
파일 하나를 같이 보낼 때 .
⚠ 본문이 33% 커집니다
(3MB 파일 → 3.8MB 로 나감)

멀티파트를 쓸 때

브라우저 <form>, curl -F,
다른 시스템이 보낼 때 .
원본 크기 그대로 나갑니다 .
상한은 .env 의 UPLOAD_MAX_BYTES

두 방식 모두 이름을 걸러내고
겹치면 -2, -3 을 붙입니다 .

정리

JSON 과 함께 보낼 땐 base64 · 파일만 보낼 땐 멀티파트 . 저장 규칙은 둘이 같습니다

로그 보기 ① — 서버가 남긴 기록 확인하기

서버가 남긴 파일을 그대로 여는 화면입니다 . 콘솔 [로그] 메뉴 → 종류 폴더 → 월 → 파일 순서예요

aidot-express 관리자 콘솔

admin

홈

대시보드

컨트롤러

서비스

SQL

API 테스트

로그

사용자

일반

SQL

general = 서버가 한 일 · sql = LOG_SQL=true 일 때 생기는 SQL 전용

① 폴더 (월별)

2026-08

2026-07

② 파일 (하루에 하나씩)

파일	크기	수정 시각
2026-08-24.log	71 KB	오늘 14:31
2026-08-23.log	224 KB	어제 23:59
2026-08-22.log	351 KB	2 일 전

③ 내용

2026-08-24 14:30:11 [INFO] [Route] GET /api/snacks/ → SnackController.list

2026-08-24 14:30:11 [INFO] SnackService::list 호출됨

2026-08-24 14:30:55 [INFO] [sse] 구독 id=sse_mt6c... channel=guestbook (총 2명)

2026-08-24 14:31:02 [WARN] [404] GET /api/snack - Not Found

순서는 언제나 같아요

- ① 종류 폴더 (general / sql)
- ② 월 폴더 (2026-08)
- ③ 파일 (하루에 하나)
- ④ 내용 보기

[INFO] 는 그냥 알림 ,
[WARN] 은 주의 ,
[ERROR] 는 진짜 문제 !

빨간 ERROR 부터 찾으세요 .

파일은 log 폴더에도 있어요 .
설치본은 트레이 메뉴의
[로그 폴더 열기] 로 바로 이동 .

순서

누를 순서 : [로그] → 종류 탭 (일반) → 폴더 (2026-08) → 파일 (오늘 날짜) → 내용 확인

로그 보기 ② — [로그 탐색] 으로 원하는 줄만

로그는 하루에도 수만 줄이 쌓여요 . [로그 탐색] 메뉴의 빠른 질의 버튼이면 한 번에 골라집니다

aidot-express 관리자 콘솔

admin

홈

대시보드

컨트롤러

서비스

SQL

API 테스트

로그

사용자

SnackService

검색

검색 결과 zip

☐ 정규식

☒ 일치 라인만

수준

전체

ERROR

WARN

INFO

12 건 일치 · 필터 후 12 줄 표시

```
2026-08-24 14:30:11 [INFO]    SnackService::list 호출됨
2026-08-24 14:30:24 [INFO]    SnackService::create 호출됨
2026-08-24 14:30:24 [ERROR]   SnackService::create 실패
ER_NO_SUCH_TABLE: Table 'aidot_express.snack' doesn't exist
2026-08-24 14:31:40 [INFO]    SnackService::list 호출됨
2026-08-24 14:32:02 [INFO]    SnackService::remove 호출됨
```

이렇게 찾아보세요

- 컨트롤러 이름
- 서비스 이름
- 주소 (/api/snacks)
- 오류 코드 (ER_ 로 시작)

[일치 라인만] 을 켜면
찾은 줄만 남아서
훨씬 보기 편해요 .

[검색 결과 zip] 을 누르면
검색 결과를 파일로 받아
다른 사람에게 보낼 수 있어요 .

순서

누를 순서 : 검색어 입력 → [일치 라인만] 켜기 → 수준을 ERROR 로 → [검색] → 필요하면 [검색 결과 zip]

로그 보기 ③ — 에러가 났을 때 찾아가는 순서

"안 돼요" 에서 멈추지 말고 , 아래 5 단계를 그대로 따라가면 원인이 나옵니다



실제 로그를 읽어 봅시다

```
2026-08-24 14:30:24 [INFO] [Route] POST /api/snacks/ → SnackController.create
2026-08-24 14:30:24 [INFO] SnackService::create 호출됨
2026-08-24 14:30:24 [ERROR] SnackService::create 실패
ER_NO_SUCH_TABLE: Table 'aidot_express.snack' doesn't exist
at SnackService.create (workspace/service/SnackService.js:38)
```

- 어디서 ?** SnackController.create → 새 간식 넣기 기능
- 무엇이 ?** Table 'snack' doesn't exist → 테이블이 없음
- 어떻게 ?** 0 단계로 돌아가 snack 테이블을 만들면 해결

기억할 것 — 로그는 위에서 아래로 시간 순서입니다 . **ERROR** 줄을 찾았으면 그 " 바로 위 몇 줄 " 이 원인을 알려 줍니다 .

파트 3 : " 누가 " 요청하고 어떻게 했는지 확인하기

14 단계 — 요청 추적 ① 누가 · 무엇을 · 어떻게

[관리 → 요청 추적] — 우리 서버에 들어온 요청 하나하나를 되짚어 봅니다 . 로그 줄을 뒤지지 않아도 돼요

기록 16,093건 · 보관 · 30일 · 최대 200,000건

보관 정리 지금 실행

원가 실패했어요
오류-느린 요청부터 봅니다

요청 번호가 있어요
화면에 뜬 번호로 바로 찾습니다

누가 뭘 했는지 몰래요
사람을 고르고 행적을 봅니다

100
요청

10
실패

287ms
p95 소요

18
서로 다른 경로

4분
기간

요청 추적

하나의 요청이 어떤 단계를 거쳤는지, 누가 무엇을 했는지 되짚습니다

요청 ID 로바로 찾기

a3f9c210 — 오류 화면에 표시된 번호

찾기 자주 찾는 것

응답 헤더 X-Request-Id 와 로그 줄의 [번호] 가 같은 값입니다.

⊗ 서버 오류(5xx)

⚠ 실패한 요청(4xx+)

⌵ 1초 넘는 요청

🗑 삭제 행위

사용자

메서드

경로 포함

상태 ≥

소요 ≥ (ms)

☒ 내 요청만

전체

/api/admin/users

400

500

admin

🔍

요청 목록 100

시각	요청	상태	소요	이유
10:31:26	GET /api/admin/trace ↳ admin · 1d256c5c	200	30ms	
10:31:26	GET /api/admin/trace/status ↳ admin · 854faa01	200	13ms	

세 가지 카드 중 하나만 고르면 돼요

뭔가 실패했어요 → 오류 · 느린 요청부터

요청 번호가 있어요 → 번호로 바로

누가 뭘 했는지 몰래요 → 사람별 행적

처음 열면 "내 요청" 이 보여요

위쪽 숫자는 요청 수 · 실패 · p95 소요 · 경로 수 .

[내 요청만] 스위치를 끄면 다른 사람 것도 보여요 .

자주 찾는 것 4 개

[서버 오류 (5xx)] [실패한 요청 (4xx+)]

[1 초 넘는 요청] [삭제 행위]

누르면 조건이 채워지고 목록이 바뀝니다 .

순서

왼쪽 [요청 추적] → 카드 하나 고르기 → [실패한 요청 (4xx+)] 눌러 보기 → 목록에서 한 줄 클릭

10:31:25	↳ admin · 6581da6c	200	5ms
----------	--------------------	-----	-----

14 단계 — 요청 추적 ② 한 줄을 누르면 " 이야기 " 가 펼쳐져요

목록에서 POST /api/books 를 누르면 오른쪽에 그 요청의 이야기와 단계가 나옵니다

요청 목록 24

시간	요청	상태	소요	이유
10:31:19	DELETE /api/books/:id student1-7953f1e7	200	4ms	
10:31:19	GET /api/books/:id student1-ecfc1915	200	5ms	
10:31:19	PUT /api/books/:id student1-054bc965	200	4ms	

d0b0adb6 이 요청의 로그 보기 201 이야기 폭포수 원자료

student1님이 책을 만들었습니다 — 성공 (6ms, SQL 1회, 1건)
2026. 8. 27. 오후 10:31:19 · 127.0.0.1 · trace 61a1a5ed3...

+0ms POST /api/books 요청이 들어왔습니다
+1ms BookController.create가 요청을 받았습니다
+3ms SQL book:insert 실행 (1ms) → 1건
+4ms BookService.create를 호출했습니다
+6ms BookController.create가 처리를 마쳤습니다 (5ms)

이야기 = 한 문장 요약 "student1 님이 책을 만들었습니다 — 성공 (6ms, SQL 1 회 , 1 건)"
누가 · 무엇을 · 결과가 어땠는지가 한 줄에 있어요 . 실패했으면 이유가 여기 적혀요 .

단계 = 요청이 지나간 길

+0ms 요청이 들어옴 → +1ms BookController.create → +3ms SQL book:insert (1ms, 1 건) → +3ms BookService → +6ms 처리 끝
어디서 오래 걸렸는지 , 어떤 SQL 이 몇 건을 다뤘는지 보여요 .

보기 3 가지

[이야기] 사람 말로 · [폭포수] 시간 막대 · [원자료] JSON 그대로
오른쪽 위 초록 숫자는 HTTP 상태 (201).

실습

- ① API 테스트로 간식 (또는 책) 을 하나 만든다
- ② [요청 추적] 목록에서 POST 줄을 누른다
- ③ 이야기와 단계를 읽어 본다

왜 좋아요 ?

" 안 돼요 " 라고 하면 예전엔 로그 파일을 통째로 뒤졌어요 .
이제 그 사람의 그 요청을 찾아 어디서 멈췄는지 바로 봅니다 .

실습

목록의 **POST /api/books** 클릭 → 이야기 읽기 → [폭포수] 눌러 시간 막대 보기 → [원자료] 로 JSON 확인

14 단계 — 요청 추적 ③ 누가 뭘 했는지 — 사람 단위로 추적

[누가 뭘 했는지 볼래요] 카드 → 사람을 고르면 그 사람의 행적이 시간순으로 펼쳐집니다

 student1님의 행적

닫기

🕒 2026. 8. 27. 오후 10:28:00 — 오후 10:31:19 24건

- 오후 10:28:00 • student1님이 책을 조회했습니다 — 성공 (9ms)
- 오후 10:28:00 • student1님이 책을 조회했습니다 — 성공 (9ms)
- 오후 10:28:00 • student1님이 책을 만들었습니다 — 성공 (6ms)
- 오후 10:28:00 • student1님이 책 한건을 수정했습니다 — 성공 (4ms)
- 오후 10:28:00 • student1님이 책 한건을 조회했습니다 — 성공 (6ms)
- 오후 10:28:00 • student1님이 책 한건을 삭제했습니다 — 성공 (4ms)
- 오후 10:29:14 • student1님이 책을 조회했습니다 — 성공 (11ms)
- 오후 10:29:14 • student1님이 책을 조회했습니다 — 성공 (6ms)

어떻게 여나요 ?

- ① 카드 [누가 뭘 했는지 볼래요]
- ② " 사람으로 찾기 " 에서 이름 (student1) 을 누른다
- ③ 오른쪽에 "student1 님의 행적 " 이 뜬다

묶음으로 보여요

" 오후 10:28:00 — 10:31:11 · 18 건 " 처럼 한 번에 한 일을 묶어요 . 30 분 넘게 쉬면 다른 묶음 .
행적 한 줄을 누르면 그 요청의 상세가 열려요 .

실습 준비 — 친구 계정

[사용자] 메뉴에서 student1 (역할 user) 을 만들고 , 그 계정으로 책을 만들고 고치고 지워 보세요 . 그다음 여기서 student1 을 골라요 .

순서

[누가 뭘 했는지 볼래요] → 사람으로 찾기 : student1 → 행적 묶음 읽기 → 한 줄 클릭 → 상세

14 단계 — 요청 추적 ④ 요청 번호로 찾기 · 로그와 잇기

오류 화면 · 응답 헤더 · 로그 줄에 찍힌 8 자리 번호 하나로 어디서든 같은 요청을 찾습니다

요청 추적 하나의 요청이 어떤 단계를 거쳤는지, 누가 무엇을 했는지 되짚습니다

요청 ID 로바로 찾기

d0b0adb6 찾기 자주 찾는 것

응답 헤더 X-Request-Id 와 로그 줄의 [번호] 가 같은 값입니다. 서버 오류(5xx) 실패한 요청(4xx+) 1초 넘은 요청 식제 행위

사용자 ☐ 내 요청만 ☐ 전체 메시지 경로 포함 상태 ≥ 소요 ≥ (ms)

student1 /api/books 400 500 🔍

요청 목록 24 📄 d0b0adb6 🔍 이 요청의 로그 보기 201 📖 이야기 📊 폭포수 📄 원자료

🔍 거르기 14,277줄 · 1파일

📄 4줄 npm run logs d0b0adb6

유형

app	11,455
http	1,996
sql	577
migration	134
boot	76
auth	39

수준

INFO	10,876
DEBUG	3,228
WARN	149
ERROR	24

요청 많은 순

```
22:31:19.774 [INFO] app d0b0adb6 [BookController.js:33:14] BookController::create 호출됨 -> params={"title"...
22:31:19.775 [INFO] app d0b0adb6 [BookService.js:30:14] BookService::create 호출됨
22:31:19.776 [DEBUG] sql d0b0adb6 [db.js:338:36] [DB:mariadb] INSERT INTO sample.book (title, author... 1.2ms
22:31:19.780 [DEBUG] http d0b0adb6 [server.js:129:7] 127.0.0.1 user=- "POST /api/books HTTP/1.1" 201 136 "...
```

① [요청 번호가 있어요] → 번호 입력 → [찾기]

② 상세의 [이 요청의 로그 보기] → 로그 탐색이 그 번호로만 좁혀짐

번호는 어디에 있나요 ?

- 응답 헤더 X-Request-Id
 - 오류 화면의 [번호]
 - 로그 줄 앞의 [번호]
 - 요청 추적 목록의 회색 번호
- 전부 같은 번호예요 .

두 화면이 이어져요

요청 추적 → [이 요청의 로그 보기] → 로그 탐색 (그 요청 줄만)
로그 탐색의 줄 옆 📄 아이콘 → 다시 요청 추적 상세
단계에 안 보이는 것 (서비스 안의 log.info) 은 로그에서 봐요 .

실습

- ① API 테스트에서 아무 요청을 보내고 응답 헤더 X-Request-Id 를 복사
- ② [요청 번호가 있어요] 에 붙여 넣고 [찾기]
- ③ [이 요청의 로그 보기] 로 로그까지

순서

번호 복사 → [요청 번호가 있어요] → [찾기] → [이 요청의 로그 보기] → 로그 줄의 📄 로 되돌아오기

14 단계 — 요청 추적 ⑤ 안 보일 때 · 알아 둘 것

요청이 목록에 없다고 놀라지 마세요 — 이유는 보통 이 넷 중 하나예요

증상	이유	이렇게
내 GET 요청이 목록에 없어요	로그인 안 한 (익명) 빠른 GET 은 DB 에 저장하지 않고 메모리에만 잠깐 둡니다	[요청 번호가 있어요] 로는 찾혀요 (최근 1,000 건). 오류 · 느린 것 · 로그인 사용자 요청은 저장돼요
다른 사람 요청이 안 보여요	[내 요청만] 스위치가 켜져 있어요	스위치를 끄고 사용자 칸에 이름을 적어요
위에 노란 띠가 떠요	서버가 바빠서 잠시 기록을 줄이고 있어요	조금 뒤 다시 보면 됩니다 — 오류 · 느린 요청은 그동안에도 남아요
[이 요청의 로그 보기] 가 비어요	관리자 API 의 요청 줄은 로그에 숨겨 두고 , SQL 이 없는 빠른 요청은 남길 줄이 없어요	단계는 [요청 추적] 상세에 있어요 . 로그까지 보려면 .env 에 LOG_ADMIN=true
서버를 껐다 켜는데 남아 있나요 ?	저장된 요청은 DB(request_traces) 에 30 일 보관	상단 띠의 [보관 정리 지금 실행] 으로 오래된 것을 지울 수 있어요

한 줄로 : " 누가 · 무엇을 · 어떻게 " 는 [요청 추적], " 그때 서버가 무슨 말을 했나 " 는 [로그 탐색]. 번호 하나로 둘 사이를 오갑니다 .

정리

요청 추적 = 요청 단위의 이야기와 단계 · 사람 단위 행적 · 번호로 찾기 · 로그와 왕복

잘 안 될 때 — 이것부터 보세요

거의 모든 문제는 아래 8 가지 중 하나예요

404 Not Found

주소가 없어요

/api/snacks 뒤에 / 를 빠뜨렸는지 , 철자가 맞는지 확인

500 + Table doesn't exist

테이블이 없어요

0 단계 CREATE TABLE 을 실행했는지 확인

500 + Unknown column

컬럼 이름이 달라요

SQL 의 이름과 테이블의 컬럼 이름을 맞춰 주세요

빈 목록 []

자료가 없어요

정상 ! POST 로 간식을 한 개 넣어 보세요

401 Unauthorized

로그인이 필요해요

컨트롤러에서 [전체 인증 필요] 를 체크했는지 확인

화면이 안 열려요

서버가 꺼졌어요

검은 창에 npm start 가 켜져 있는지 확인

CORS 오류 (빨간 글씨)

주소가 달라서 막힘

Vue 는 vite.config.js 의 프록시 확인 · 배포본은 CORS_ORIGIN 설정

SSE 소식이 안 와요

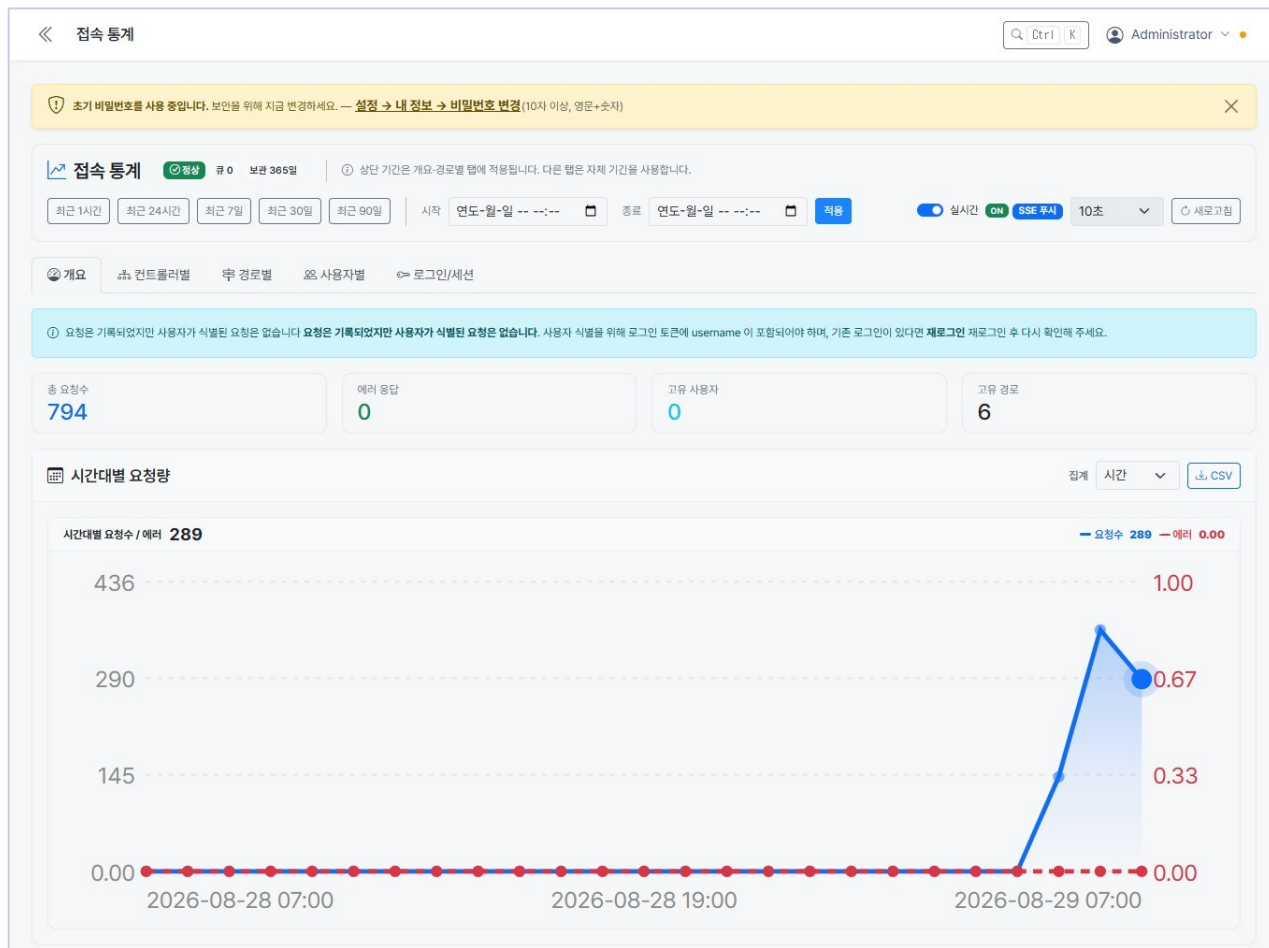
연결이 안 됐어요

● 표시가 초록인지 , /api/events/stats 에 내가 보이는지 확인

그래도 모르겠으면 콘솔 왼쪽 메뉴 [로그] 에서 빨간 줄을 찾아보세요 . 무엇이 잘못됐는지 한국어로 적혀 있어요 .

15 단계 ① — 누가 얼마나 썼는지 보기

왼쪽 메뉴 아래쪽 [접속 통계] 를 누르면 지금까지의 요청이 모두 모여 있습니다



맨 위 기간 단추

[최근 1 시간] · [최근 24 시간] · [최근 7 일] ...

직접 날짜를 넣으려면 시작 · 종료를 고르고 [적용].

[실시간 ON] 을 켜면

숫자가 저절로 새로고침됩니다 .

옆의 10 초는 갱신 간격이에요 .

탭이 다섯 개

개요 · 컨트롤러별 · 경로별 ·

사용자별 · 로그인 / 세션

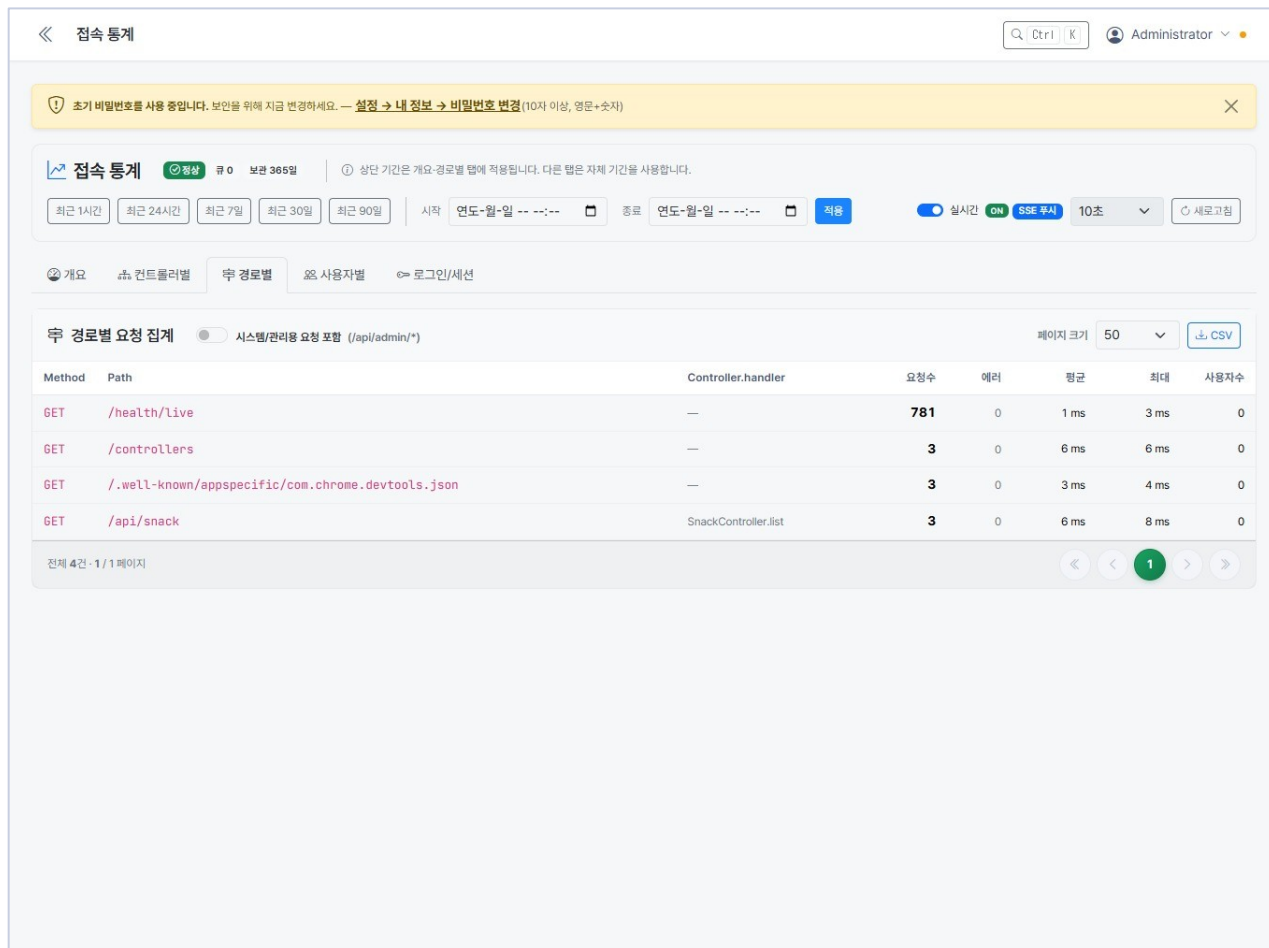
개요에는 요청 수와 시간대별 그래프가 나옵니다 .

누르기

왼쪽 메뉴 [접속 통계] → 기간 고르기 → 탭 넘겨 보기

15 단계 ② — 어떤 기능이 얼마나 불렀나

[컨트롤러별] 은 기능 단위 , [경로별] 은 주소 단위로 보여 줍니다



컨트롤러별

SnackController 787 건 처럼

" 어떤 기능을 얼마나 썼나 " 가
한 줄로 보입니다 .

시스템 요청은 빠져 있어요

헬스체크 · 콘솔 화면 같은 것은
세지 않습니다 . 보고 싶으면
[시스템 요청 포함] 을 켜세요 .

[CSV] · [엑셀] 로

지금 보이는 표를 그대로
내려받을 수 있어요 .

읽는 법

요청수 · 에러 (5xx) · 4xx · 평균 / 최대 응답시간 · 사용자수 — 느린 기능은 평균이 큼니다

15 단계 ③ — [사용자별] 이 비어 있나요 ?

누가 불렀는지 남기려면 컨트롤러에 인증이 켜져 있어야 합니다

왜 비어 있을까

지금까지 만든 /api/snack 은 아무나 부를 수 있게 열어 두었습니다 .
누가 불렀는지 알 수 없으니 사용자별에 넣을 수가 없어요 .

이렇게 하면 사람 이름이 남습니다

① 컨트롤러를 열고

[컨트롤러] → SnackController 의 연필 아이콘

② [전체 인증 필요] 스위치를 켜다

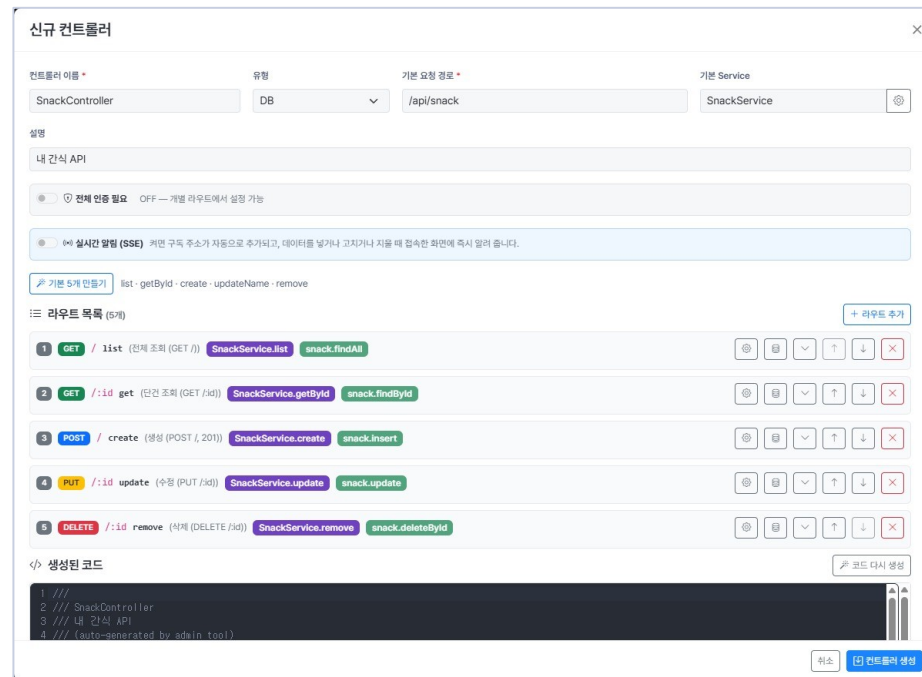
켜면 모든 라우트가 로그인한 사람만 부를 수 있게 됩니다

③ [컨트롤러 생성 / 저장]

코드에 @Auth() 가 붙습니다

④ 부를 때 토큰을 함께 보낸다

[API 테스트] 는 [토큰 자동 첨부] 가 켜져 있으면 알아서 붙여 줍니다



```
@Controller('/api/snack')
@Auth() // ← 이 줄이 생깁니다
export default class SnackController { ... }
```

핵심

인증을 켜야 [사용자별] 에 이름이 남습니다 — 토큰 없이 부르면 " 이름 없음 " 으로 씁입니다

15 단계 ④ — 사용자별 통계 뽑기

한 사람이 무엇을 얼마나 썼는지 보고, 엑셀로 내려받습니다

접속 통계

Administrator

초기 비밀번호를 사용 중입니다. 보안을 위해 지금 변경하세요. — 설정 → 내 정보 → 비밀번호 변경 (10자 이상, 영문+숫자)

접속 통계

0

보관 365일

상단 기간은 개요-경로별 탭에 적용됩니다. 다른 탭은 자체 기간을 사용합니다.

최근 1시간

최근 24시간

최근 7일

최근 30일

최근 90일

시작 연도-월-일 -- --:--

종료 연도-월-일 -- --:--

적용

실시간 ON SSE 무시 10초

새로고침

개요

컨트롤러별

루트 경로별

사용자별

로그인/세션

컨트롤러별

사용자 아이디 (선택)

콘솔 제외

CSV

엑셀

컨트롤러	호출	라우트	사용자	4xx	5xx	평균(ms)	최대(ms)	마지막 호출
(기타)	787	3	0	3	0	1	6	2026-08-29 07:48:12
SnackController	3	1	0	0	0	6	8	2026-08-29 07:47:12

【 사용자별 】 탭

사람마다 요청수 · 에러 ·
평균 응답 · 처음 / 마지막 시각이
한 줄로 나옵니다 .

한 사람만 보려면

【 컨트롤러별 】 탭 위의
사용자 아이디 칸에 이름을 넣으세요 .
그 사람이 쓴 기능만 보입니다 .

【 엑셀 】 을 누르면

보고 있는 기간 · 조건 그대로
.xlsx 파일로 받습니다 .

내려받기 기간 고르기 → (필요하면 아이디 입력) → 【 엑셀 】 또는 【 CSV 】

15 단계 ⑤ — 한 사람이 무엇을 했는지 따라가기

통계는 "얼마나", 추적은 "무엇을 언제" — 셋을 이어서 봅니다

① 언제 들어왔나 [접속 통계] → [로그인 / 세션]

로그인 · 로그아웃 시각, 어느 IP 에서 들어왔는지.
이상하면 그 자리에서 [강제 종료] 로 끊을 수 있어요.

② 무엇을 불렀나 [접속 통계] → [사용자별] → 이름 클릭

그 사람이 부른 경로가 많이 부른 순서로 나옵니다.
(같은 화면의 [원본 로그] 는 한 줄씩 시간순)

③ 그때 무슨 일이 있었나 [요청 추적]

요청 하나를 골라 컨트롤러 → 서비스 → SQL 까지
어디서 몇 ms 가 걸렸는지 단계별로 봅니다.

세 화면이 같은 기록을 본다

접속 통계 = 모아서 센 것
원본 로그 = 한 줄씩
요청 추적 = 그 요청 안에서 일어난 일
보관 기간은 접속 통계 365 일,
요청 추적 30 일이 기본입니다.

감사 자료로 낼 때

기간을 정하고 [엑셀] 로 받으면
시트 하나로 정리되어 나옵니다.
사용자 아이디를 넣으면 그 사람 것만.

⚠ 인증을 켜지 않은 API 는 이름이 남지 않습니다

순서

로그인 / 세션 (언제) → 사용자별 (무엇을) → 요청 추적 (그 안에서 무슨 일이)

16 단계 ① — 인증을 컨 API 는 어떻게 부르나

로그인해서 받은 " 토큰 " 을 요청에 붙이면 됩니다 . 그래야 누가 불렀는지도 남습니다

토큰이 뭔가요

출입증 같은 것입니다 . 로그인하면 서버가 긴 글자를 하나 줍니다 .
그 뒤로는 요청마다 이 글자를 같이 보내면 서버가 " 아 , 아무개구나 " 하고 알아봅니다 .
유효기간은 15 분이고 , 지나면 다시 로그인하거나 갱신하면 됩니다 .

Bearer 를 빠뜨리지 마세요

Authorization 값은

Bearer + 공백 + 토큰 입니다 .
토큰만 넣으면 통과되지 않습니다 .

① 로그인해서 토큰 받기

POST /api/admin/auth/login

```
{ "username": "admin", "password": "admin1234" }
```

→ data.accessToken 에 긴 글자가 들어 있습니다

왜 굳이 켜나요

- 아무나 부르지 못하게 막습니다
- [접속 통계] → [사용자별] 에 이름이 남습니다
- 문제가 생겼을 때 누가 무엇을 했는지 볼 수 있습니다

② 요청에 붙이기 — 헤더 한 줄

GET /api/snack

Authorization: Bearer eyJhbGciOi...

붙이지 않으면 401 (권한 없음) 이 돌아옵니다

토큰은 비밀번호와 같습니다

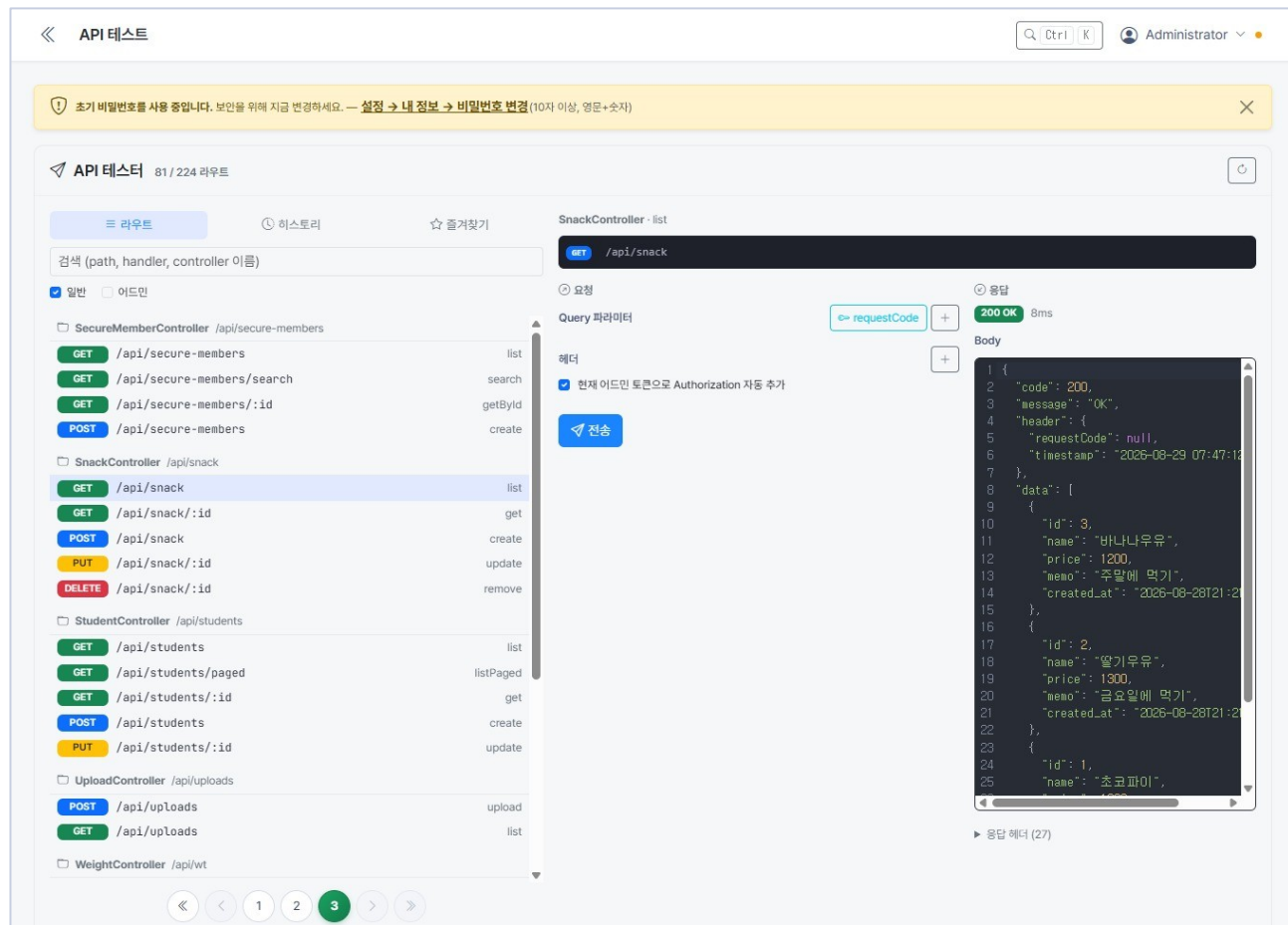
메신저 · 이슈에 붙여넣지 마세요 .
만료가 15 분이라 오래 못 쓰지만 ,
그 사이에는 그 사람 행세를 할 수 있습니다 .

핵심

로그인 → accessToken 받기 → 요청 헤더에 Authorization: Bearer < 토큰 >

16 단계 ② — 콘솔 [API 테스트] 로 부르기

토큰을 직접 복사할 필요가 없습니다 — 켜 두면 알아서 붙여 줍니다



① 왼쪽 [API 테스트]

만든 컨트롤러가 목록에 나옵니다

② 부를 라우트를 고른다

[GET] /api/snack → list

③ [토큰 자동 첨부] 를 켜다

지금 로그인한 사람의 토큰이 붙습니다

④ [전송]

아래에 응답이 그대로 보입니다

401 이나오면

- [토큰 자동 첨부] 가 꺼져 있거나
 - 로그인이 풀렸습니다 (다시 로그인)
- 403 이면 역할 (admin/user) 이 모자란 것입니다 .

누르기

[API 테스트] → 라우트 선택 → [토큰 자동 첨부] ON → [전송]

16 단계 ③ — 포스트맨 (Postman) 으로 부르기

콘솔 밖에서 부를 때는 토큰을 직접 받아 헤더에 넣습니다

1) 먼저 로그인해서 토큰을 받습니다

```
POST http://localhost:7901/api/admin/auth/login
Body → raw → JSON
{ "username": "admin", "password": "admin1234" }
```

응답의 data.accessToken 을 복사합니다

매번 복사하기 번거롭다면

```
// 로그인 요청의 Scripts → Post-response 에
const d = pm.response.json().data;
pm.environment.set("token", d.accessToken);

// 다른 요청에서는 {{token}} 으로 씁니다
Authorization: Bearer {{token}}
```

2) 업무 API 를 부를 때 헤더에 넣습니다

```
GET http://localhost:7901/api/snack
Headers
Key    : Authorization
Value  : Bearer {{token}}
```

Authorization 탭의 Type 을 Bearer Token 으로 골라도 같습니다 .

환경 (Environment) 을 하나 만들어 두면
로그인 한 번에 나머지 요청이 모두 통과합니다 .

토큰이 15 분이면 만료됩니다
401 이 나오면 로그인 요청을 한 번 더 보내세요 .

요약

로그인 → accessToken 복사 (또는 스크립트로 저장) → Authorization: Bearer < 토큰 > 으로 호출

17 단계 ① — 토큰은 어디서 나오나요 ?

로그인하면 서버가 긴 글자를 줍니다 . 그걸 꺼내서 쓰는 거예요

1 로그인 요청을 보냅니다

```
POST /api/admin/auth/login
Content-Type: application/json

{ "username": "admin", "password": "admin1234" }
```

2 응답 안에 토큰이 들어 있습니다

```
{
  "code": 200,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ", ← 이것 !
    "refreshToken": "...",
    "user": { "username": "admin", "role": "admin" }
  }
}
```

꺼내는 방법 (도구별)

```
// 브라우저 · Node
const r = await fetch('/api/admin/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username, password }) });
const token = (await r.json()).data.accessToken;
```

```
# 포스트맨 — 로그인 요청의 Scripts → Post-response
const d = pm.response.json().data;
pm.environment.set("token", d.accessToken);
```

토큰은 비밀번호와 같습니다

메신저 · 이슈에 붙여넣지 마세요 . 15 분 뒤 만료됩니다 .

요약

로그인 → 응답의 `data.accessToken` 을 꺼내 둔다

17 단계 ② — 꺼낸 토큰을 어디에 붙이나요 ?

요청 헤더에 한 줄 . 도구마다 붙이는 자리만 다릅니다

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

“Bearer” 와 토큰 사이에 공백이 하나 있어야 합니다 . 토큰만 넣으면 통과되지 않아요 .

콘솔 [API 테스트]

[토큰 자동 첨부] 스위치를 켜면
지금 로그인한 사람의 토큰이
저절로 붙습니다 .

가장 쉬운 방법이에요 .

포스트맨

Authorization 탭 →
Type 을 Bearer Token 으로
토큰 칸에 붙여넣기

또는 {{token}} 변수 사용

코드에서

```
fetch(url, { headers: {  
  Authorization:  
    `Bearer ${token}` } })
```

axios 도 headers 에 같은 줄

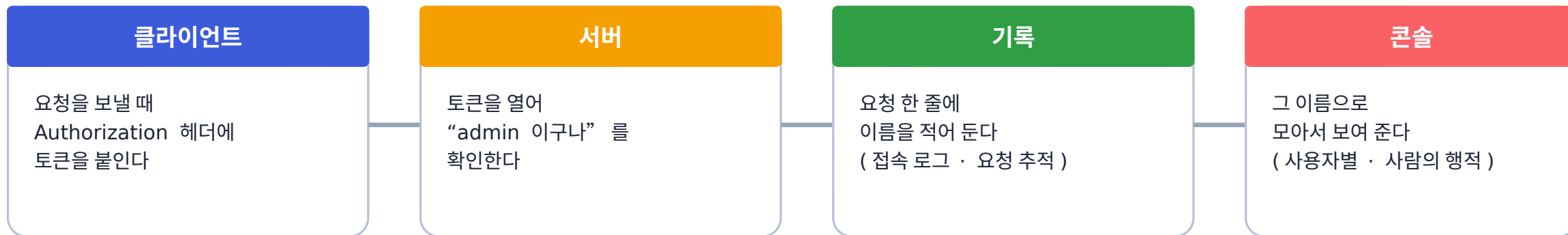
401 이 나오면 헤더를 빠뜨렸거나 토큰이 만료된 것 • **403** 이면 토큰은 맞지만 역할 (admin/user) 이 모자란 것

붙이는 자리

모든 요청의 헤더에 **Authorization: Bearer < 토큰 >**

17 단계 ③ — 그래서 콘솔이 “누가 했는지” 를 어떻게 아나요 ?

토큰 안에 이름이 들어 있고 , 서버가 요청마다 그 이름을 기록합니다



```
# 서버가 남기는 한 줄 ( 접속 로그 )
2026-08-30 07:12:03 admin GET /api/snack 200 6ms 10.0.0.31
    ↑ 토큰에서 알아낸 이름 — 이게 있어야 “누가” 를 셀 수 있습니다
```

토큰을 안 붙이면 ?

이름 칸이 비어 “이름 없음” 으로 쌓입니다 — 누가 했는지 알 수 없어요 .

로그인 자체도 기록됩니다

로그인은 토큰을 받기 전이라 , 성공한 경우에만 아이디를 기록합니다 .

한 줄

토큰을 붙이면 → 서버가 이름을 알고 → 기록에 남고 → 콘솔에서 사람으로 묶여 보인다

17 단계 ④ — 콘솔에서 사람을 따라가기

두 화면을 이렇게 씁니다 (실제 배치를 그려 놓았어요)

Aidot

대시보드

컨트롤러

접속 통계

요청 추적

로그

접속 통계

개요

컨트롤러별

경로별

사용자별

사용자

요청

에러

평균

마지막

admin	214	1	6ms	07:12
nurse01	52	0	8ms	07:05

↑ 사람을 누르면 그 사람의 경로 목록이 아래에 나옵니다

GET /api/snack

128 회

POST /api/snack

12 회

[] 시스템 요청 포함 ← 로그인 등

Aidot

대시보드

컨트롤러

접속 통계

요청 추적

로그

요청 추적 — admin 님의 행적

컨트롤러 요청

시스템 접근

전체

07:12:03

GET /api/snack

200

6ms

07:12:01

POST /api/snack

201

14ms

07:11:58

GET /api/snack/3

200

5ms

07:11:40

GET /api/snack

200

7ms

↑ 위로 올리면 더 옛날 기록을 이어서 불러옵니다

를 누르면 그 요청 하나의 단계별 상세

접속 통계 = “ 얼마나 썼나 ”

요청 추적 = “ 무엇을 언제 했나 ”

쓰는 순서

접속 통계에서 사람을 찾고 → 요청 추적에서 그 사람의 행적을 추적합니다

오늘 한 일

- ✓ 설치하고 (폴더 / 설치본) 테이블 (snack) 을 만들었어요
- ✓ 콘솔에서 SQL → 서비스 → 컨트롤러로 주소 5 개를 열었어요
- ✓ 브라우저 · API 테스트 · Postman 으로 확인했어요
- ✓ 같은 것을 파일 3 개로 직접 만들어 봤어요
- ✓ HTML · Vue 3 화면과 SSE 실시간까지 붙여 봤어요
- ✓ 페이지 나눠 보기 (executeList) 와 파일 업로드 기능을 추가해 봤어요

다음엔 이런 걸 해 보세요

검색 붙이기

목록 SQL 에 WHERE :kw 를 더하고 Query Form 위젯 연결

안전하게 잠그기

컨트롤러의 [전체 인증 필요] 체크 → 로그인해야 열림

올린 파일 관리하기

업로드 목록 화면 · 지우기 · 이미지 미리보기 붙이기

묶어서 테스트 · 백업

[시나리오 테스트] 로 한 번에 점검 · [백업 / 복원] 으로 zip 다운로드